



1. Given

x_1
 x_2
 w_1
 w_2
 b

→

$$z = x_1 w_1 + x_2 w_2 + b \longrightarrow a = g(z) \longrightarrow L = -(y \log(a) + (1 - y) \log(1 - a))$$

Sigmoid $a = g(z) = \frac{1}{1 + e^{-z}}$

Cross Entropy (CE) Loss

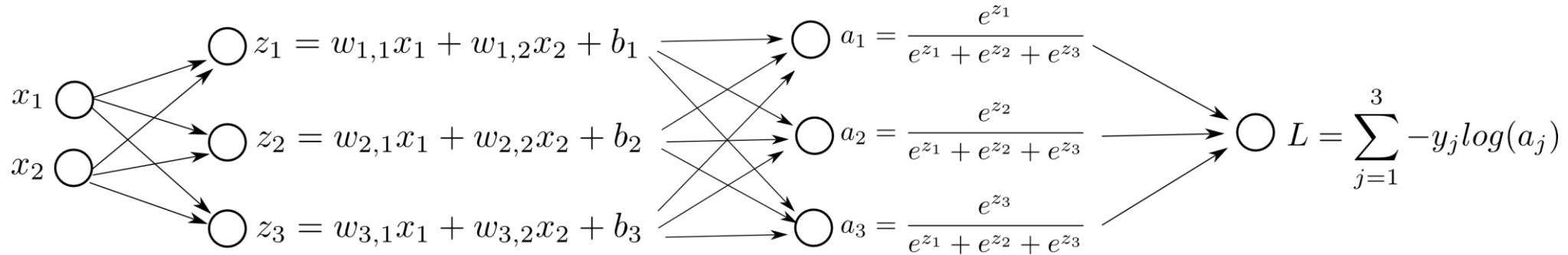
$$J = - \sum_{i=1}^N \sum_{j=1}^C y_j^{(i)} \log(a_j^{(i)})$$

C : no. Classes; N : no. Datapoints

Compute Gradient ∇L ($\frac{\partial L}{\partial W}$ and $\frac{\partial L}{\partial b}$)



2. Given



Parameters:

$$W = \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

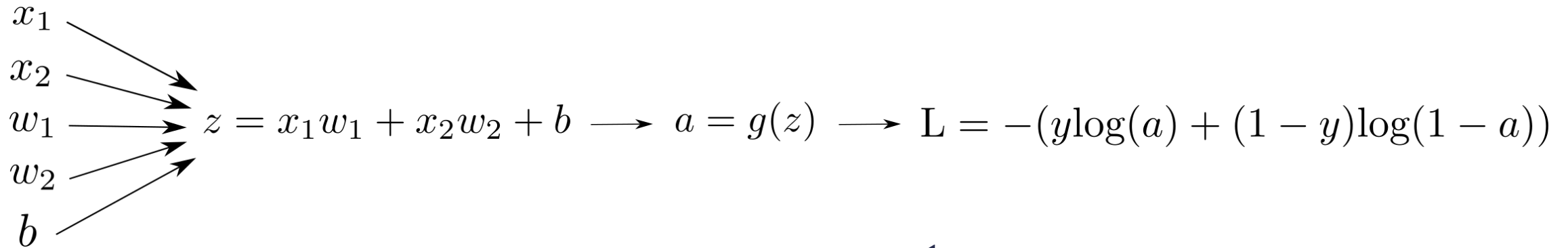
$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

$$\mathbf{a} = \text{softmax}(\mathbf{z})$$

Compute Gradient ∇L ($\frac{\partial L}{\partial W}$ and $\frac{\partial L}{\partial b}$)



Câu 1



$$z = w_1 x_1 + w_2 x_2 + b \quad a = g(z) = \frac{1}{1 + e^{-z}} \quad L = -(y \log a + (1 - y) \log(1 - a))$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w_1}$$

$$\frac{\partial z}{\partial w_1} = x_1$$

$$\frac{\partial a}{\partial z} = -\frac{(1 + e^{-z})'}{(1 + e^{-z})^2}$$

$$\frac{\partial L}{\partial a} = -\left(\frac{y}{a} - \frac{1 - y}{1 - a}\right)$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w_2}$$

$$\frac{\partial z}{\partial w_2} = x_2$$

$$= \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1 + e^{-z} - 1}{(1 + e^{-z})^2}$$

$$= \frac{-y}{a} + \frac{1 - y}{1 - a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial b}$$

$$\frac{\partial z}{\partial b} = 1$$

$$= g(z) - (g(z))^2$$

$$= a(1 - a)$$



Logistic Regression



Logistic Regression

$$\text{score} = w_1x_1 + w_2x_2 + b$$

Predict the probability that the data point $x = (x_1, x_2)$ belongs to the blue class (label 1)

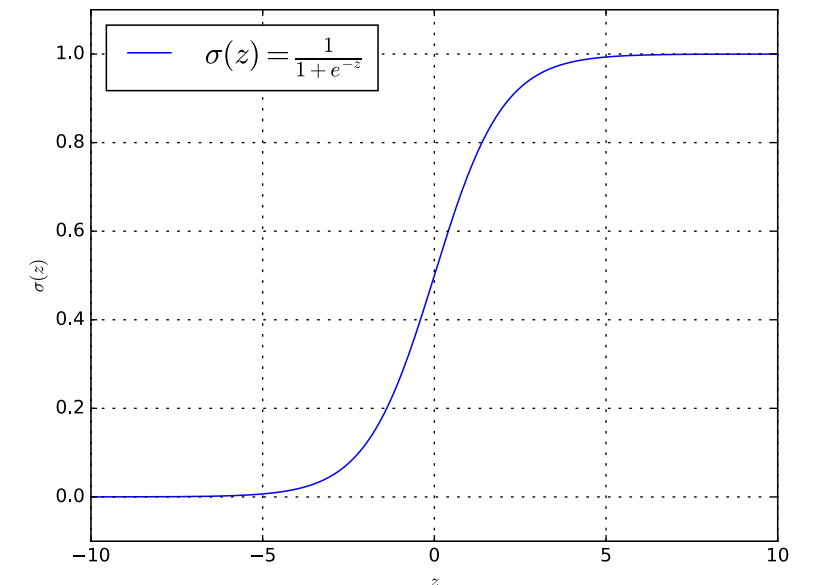
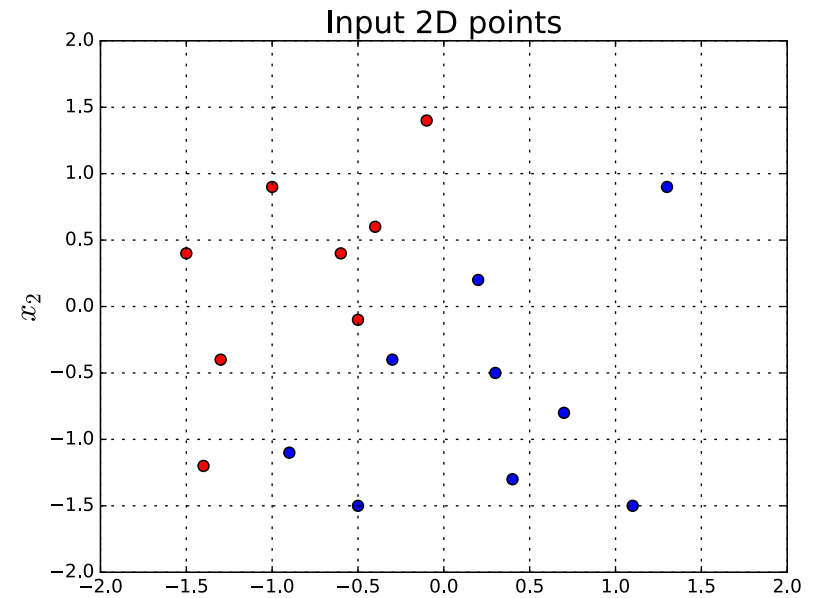
$$a = g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z = \text{score} = w_1x_1 + w_2x_2 + b = \mathbf{w}^T \mathbf{x} + b.$$

- $0 < g(z) < 1, z \in \mathbb{R}$.
- $0 < g(\mathbf{x}) < 1, \mathbf{x} \in \mathbb{R}^2$.
- a is the predicted probability by the model that the data point $\mathbf{x}$ belongs to the blue class (label 1)

The predicted label of the data point $\mathbf{x} = (x_1, x_2)$ is

$$a = \begin{cases} 0, & \text{if } g(\mathbf{x}) < 0.5 \\ 1, & \text{if } g(\mathbf{x}) \geq 0.5 \end{cases}$$





Logistic Regression

Predict the probability that the data point $x = (x_1, x_2)$ belongs to the blue class (label 1)

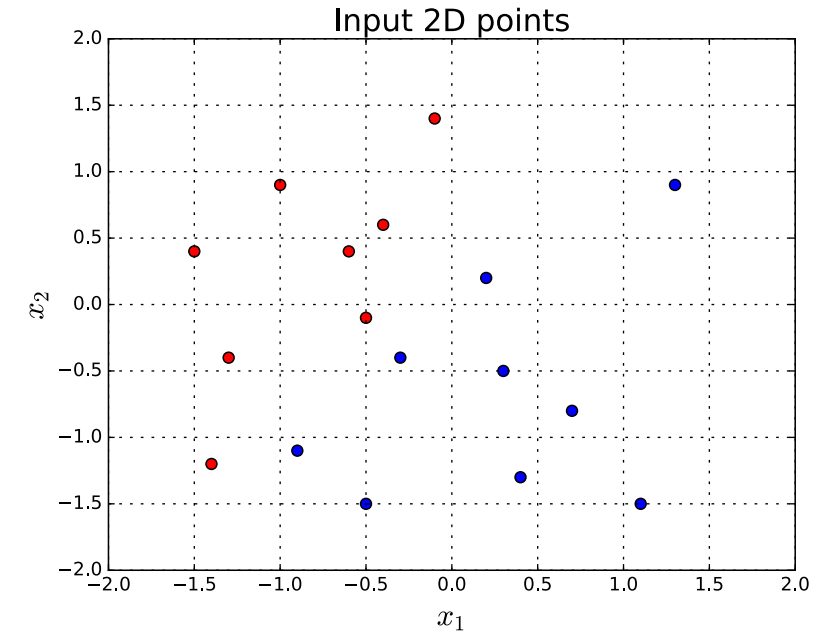
$$a = g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z = \text{score} = w_1x_1 + w_2x_2 + b = \mathbf{w}^T \mathbf{x} + b.$$

```
x = np.array([1.1, -0.6])
print(w.dot(x)+b)
print(sigmoid(w.dot(x)+b))
11.485000000000001 0.9999897169153853
```

```
x = np.array([-1.5, 0.0])
print(w.dot(x)+b)
print(sigmoid(w.dot(x)+b))
-7.505000000000001 0.0005500231583811739
```

```
x = np.array([-0.3, -0.0])
print(w.dot(x)+b)
print(sigmoid(w.dot(x)+b))
0.09099999999999997 0.5227343135939515
```



Supposing that, after training:

$$w_1 = 6.33, w_2 = -4.22, \\ b = 1.99$$

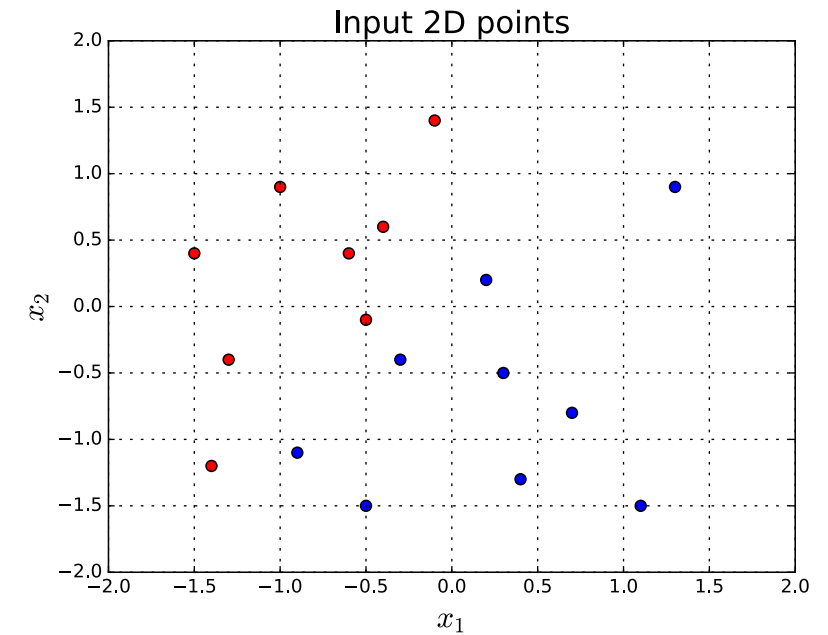


Logistic Regression – Loss Function

$$\hat{w}, \hat{b} = \operatorname{argmax}_{w, b} a^y \cdot (1 - a)^{(1-y)}$$

$$\begin{aligned}\hat{w}, \hat{b} &= \operatorname{argmax}_{w, b} \log(a^y \cdot (1 - a)^{(1-y)}) \\ &= \operatorname{argmax}_{w, b} [\log(a^y) + \log((1 - a)^{(1-y)})] \\ &= \operatorname{argmax}_{w, b} [y \log(a) + (1 - y) \log(1 - a)]\end{aligned}$$

$$\hat{w}, \hat{b} = \operatorname{argmin}_{w, b} -[y \log(a) + (1 - y) \log(1 - a)] = \operatorname{argmin}_{w, b} L(w, b)$$



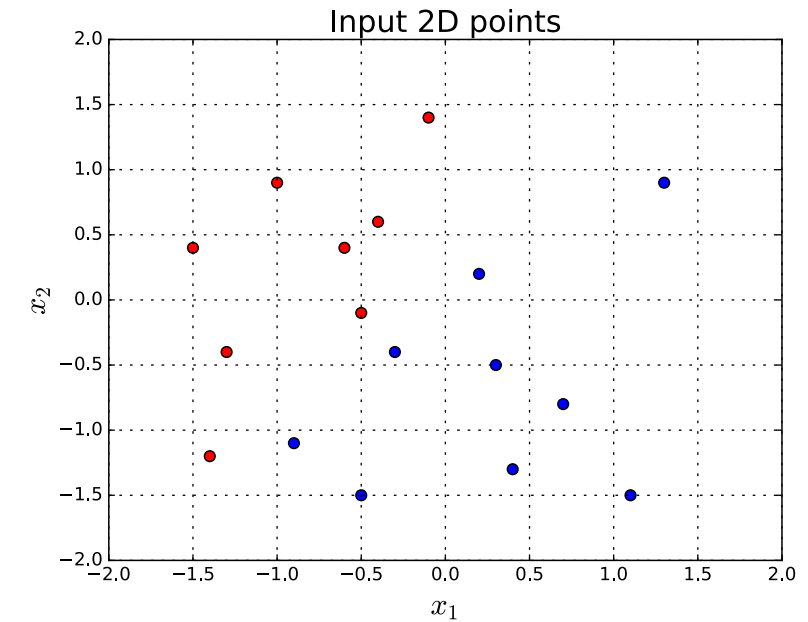
Hồi quy logistic – Ước lượng hợp lý cực đại



- All training data points N , MLE Loss function:

$$\begin{aligned}\hat{w}, \hat{b} &= \operatorname{argmax}_{w, b} \prod_{i=1}^N a^{(i) y^{(i)}} \cdot (1 - a^{(i)})^{(1 - y^{(i)})} \\ &= \operatorname{argmax}_{w, b} \log \prod_{i=1}^N a^{(i) y^{(i)}} \cdot (1 - a^{(i)})^{(1 - y^{(i)})} \\ &= \operatorname{argmax}_{w, b} \sum_{i=1}^N \left[\log(a^{(i) y^{(i)}}) + \log((1 - a^{(i)})^{(1 - y^{(i)})}) \right] \\ &= \operatorname{argmax}_{w, b} \sum_{i=1}^N [y^{(i)} \log(a^{(i)}) + (1 - y^{(i)}) \log(1 - a^{(i)})]\end{aligned}$$

$$a^{(i)} = \frac{1}{1 + e^{-z^{(i)}}} = \frac{1}{1 + e^{-(w_1 x_1^{(i)} + w_2 x_2^{(i)} + b)}}$$

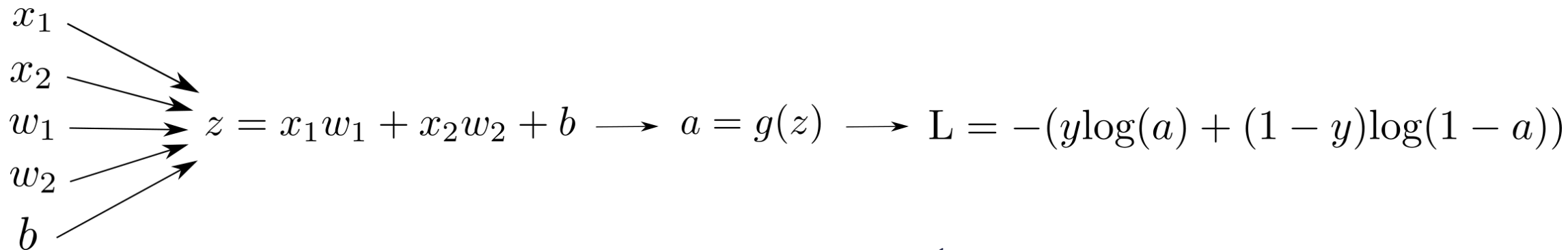




TRAINING A LOGISTIC REGRESSION MODEL



Chain rule



$$z = w_1 x_1 + w_2 x_2 + b \quad a = g(z) = \frac{1}{1 + e^{-z}} \quad L = -(y \log a + (1 - y) \log(1 - a))$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w_1}$$

$$\frac{\partial z}{\partial w_1} = x_1$$

$$\frac{\partial a}{\partial z} = -\frac{(1 + e^{-z})'}{(1 + e^{-z})^2}$$

$$\frac{\partial L}{\partial a} = -\left(\frac{y}{a} - \frac{1 - y}{1 - a}\right)$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w_2}$$

$$\frac{\partial z}{\partial w_2} = x_2$$

$$= \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1 + e^{-z} - 1}{(1 + e^{-z})^2}$$

$$= \frac{-y}{a} + \frac{1 - y}{1 - a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial b}$$

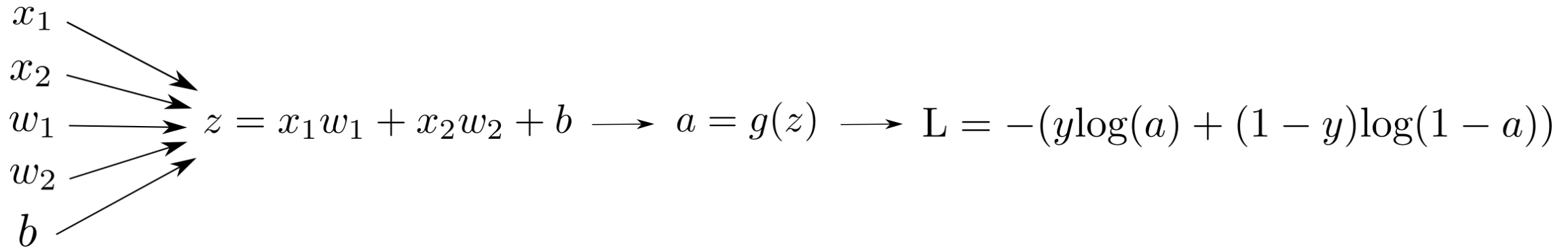
$$\frac{\partial z}{\partial b} = 1$$

$$= g(z) - (g(z))^2$$

$$= a(1 - a)$$



Chain rule



$$\begin{aligned} \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} &= \left(\frac{-y}{a} + \frac{1-y}{1-a} \right) a(1-a) \\ &= \frac{-y}{a} a(1-a) + \frac{1-y}{1-a} a(1-a) \\ &= -y(1-a) + (1-y)a \\ &= -y + ya + a - ya \\ &= a - y \end{aligned}$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w_1} = (a - y) x_1$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w_2} = (a - y) x_2$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial b} = (a - y)$$

Gradient (1 training datapoint)

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \end{bmatrix} = (a - y) \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} = (a - y) \mathbf{x}$$

Gradient (training dataset)

$$\nabla J = \sum_{i=1}^N (a^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$

Gradient Descent

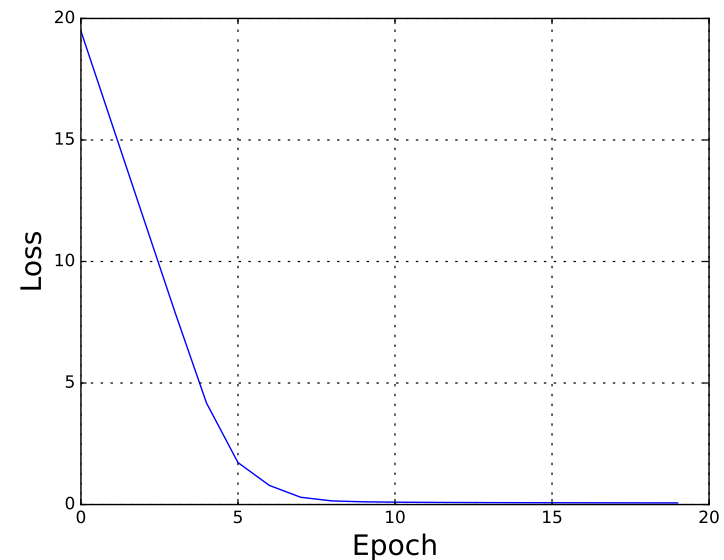
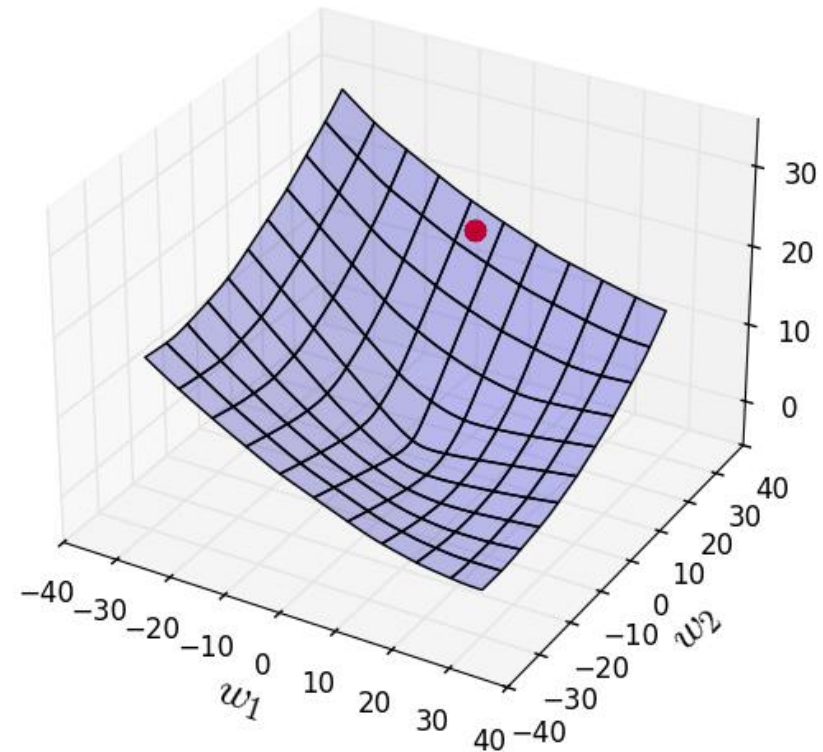
```

learning_rate = 8.0
num_epochs = 20
w = np.array([-4.0, 29.0])
b = 0.0
for epoch in range(num_epochs):
    dw = np.zeros(w.shape)
    db = 0.0
    total_loss = 0.0
    for i in range(X.shape[0]):
        # get the i-th data point and its label
        x_i = X[i,:]
        y_i = y[i]

        z_i = w.dot(x_i) + b
        a_i = sigmoid(z_i)
        loss_i = compute_loss(y_i, a_i)
        # derivatives
        dw_i = x_i * (a_i - y_i) # dL / dw_i
        db_i = a_i - y_i # dL / db

        dw += dw_i
        db += db_i
        total_loss += loss_i
    # scale the derivatives
    dw = (1.0/X.shape[0]) * dw
    db = (1.0/X.shape[0]) * db
    total_loss = (1.0/X.shape[0]) * total_loss
    print(f'Epoch {epoch+1}, Loss = {total_loss:.2f}')
    # gradient descent
    w = w - learning_rate * dw
    b = b - learning_rate * db

```





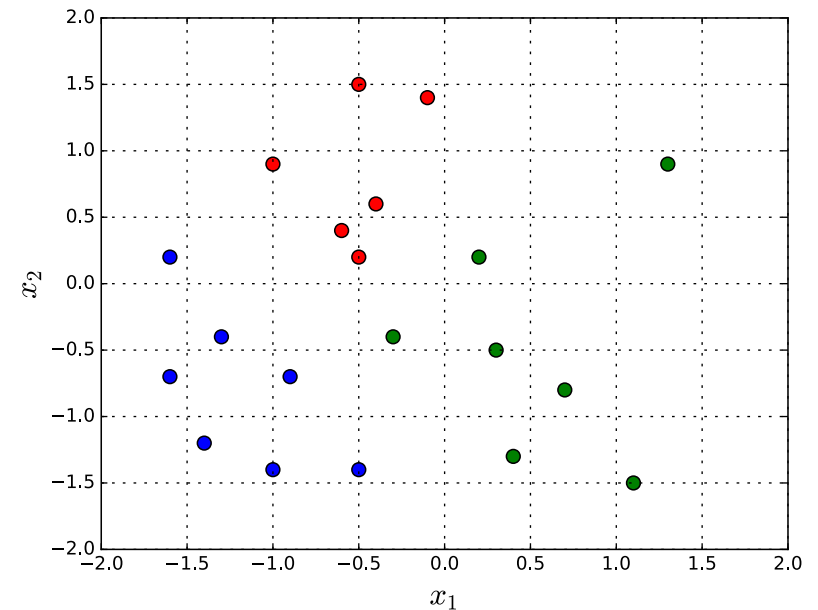
SOFTMAX REGRESSION

Classification

Predict the probability that the data point $x = (x_1, x_2)$ belongs to the red label (label 0), the green class (label 1), and the blue class (label 2)

Label $\rightarrow$ one-hot vector:

$$0 \sim \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, 1 \sim \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, 2 \sim \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$



Can we use 3 logistic regression models, to predict the probability $a_1, a_2, a_3 \in (0,1)$?

$$a_1 = \sigma(z_1) = \frac{1}{1+e^{-z_1}} \text{ với } z_1 = w_{1,1}x_1 + w_{1,2}x_2 + b_1$$

$$a_2 = \sigma(z_2) = \frac{1}{1+e^{-z_2}} \text{ với } z_2 = w_{2,1}x_1 + w_{2,2}x_2 + b_2$$

$$a_3 = \sigma(z_3) = \frac{1}{1+e^{-z_3}} \text{ với } z_3 = w_{3,1}x_1 + w_{3,2}x_2 + b_3$$

$$a_1 + a_2 + a_3 = 1????$$

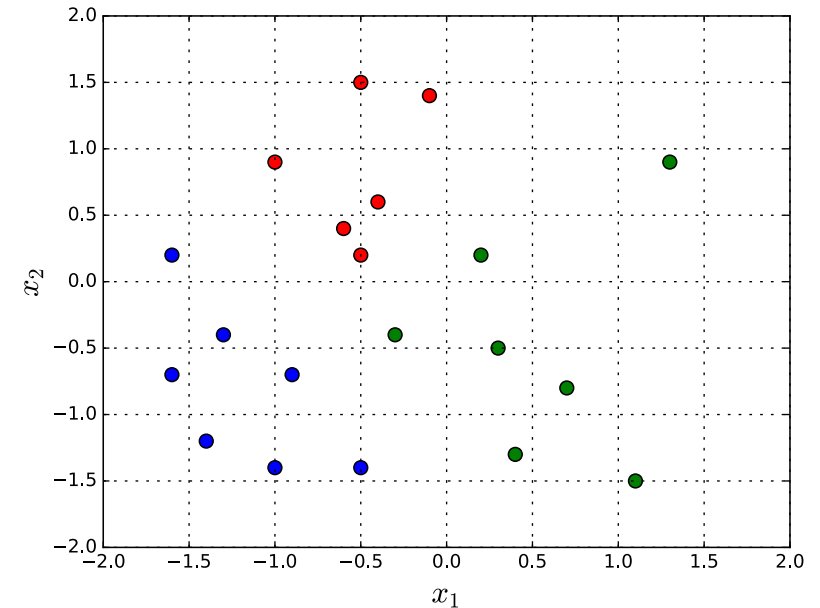
Classification

Can not constrain: $a_1 + a_2 + a_3 = 1$ with:

$$a_1 = \sigma(z_1) = \frac{1}{1+e^{-z_1}} \text{ với } z_1 = w_{1,1}x_1 + w_{1,2}x_2 + b_1$$

$$a_2 = \sigma(z_2) = \frac{1}{1+e^{-z_2}} \text{ với } z_2 = w_{2,1}x_1 + w_{2,2}x_2 + b_2$$

$$a_3 = \sigma(z_3) = \frac{1}{1+e^{-z_3}} \text{ với } z_3 = w_{3,1}x_1 + w_{3,2}x_2 + b_3$$



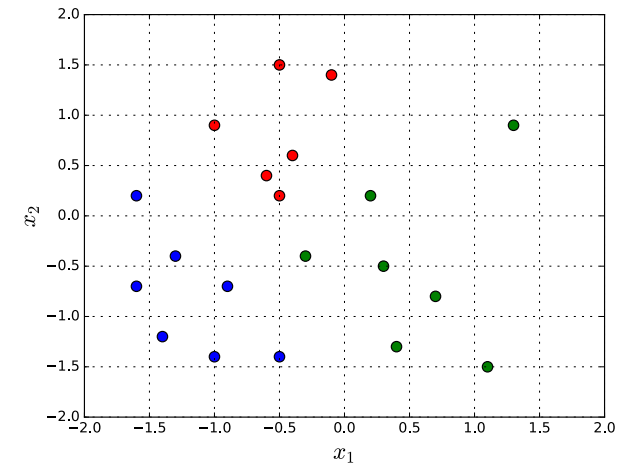
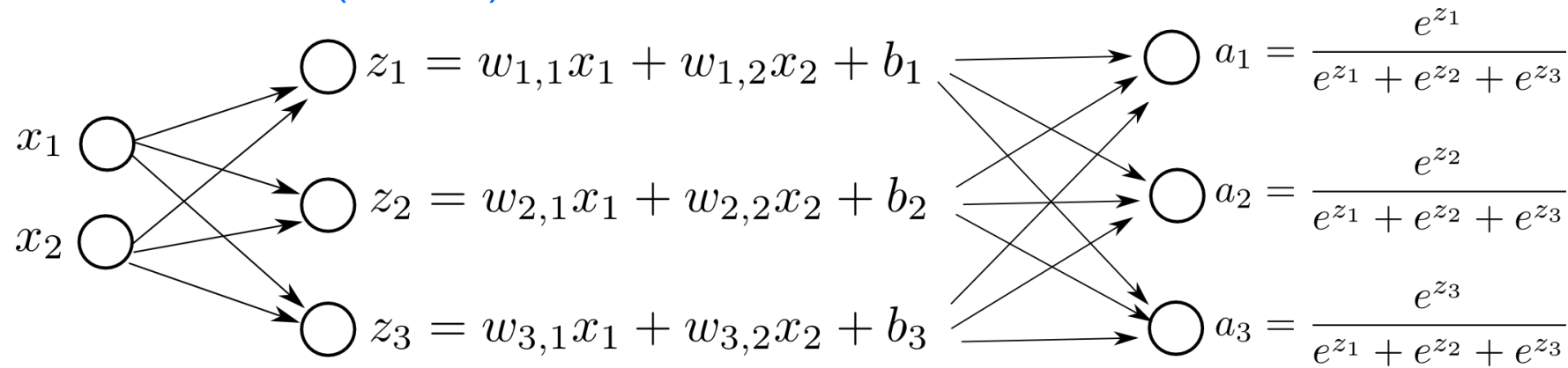
Solution: Instead of applying the logistic sigmoid function to each z_i , using the softmax function simultaneously to all z_i , $(a_1, a_2, a_3) = \text{softmax}(z_1, z_2, z_3)$.

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \quad a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \quad a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$\rightarrow a_1 + a_2 + a_3 = 1.$$

Softmax Regression

Predict the probability that the data point $x = (x_1, x_2)$ belongs to the red label (label 0), the green class (label 1), and the blue class (label 2).



Parameters

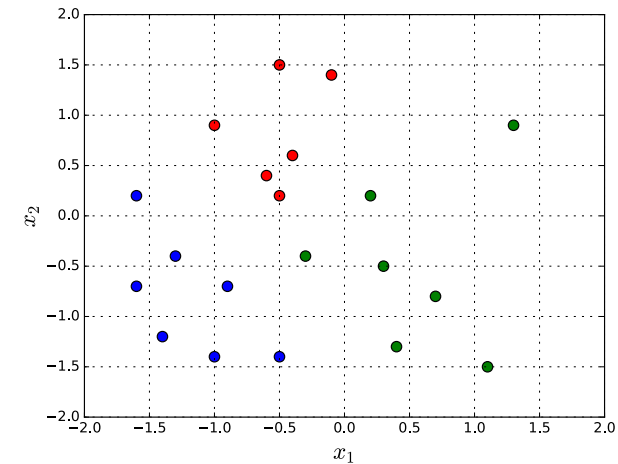
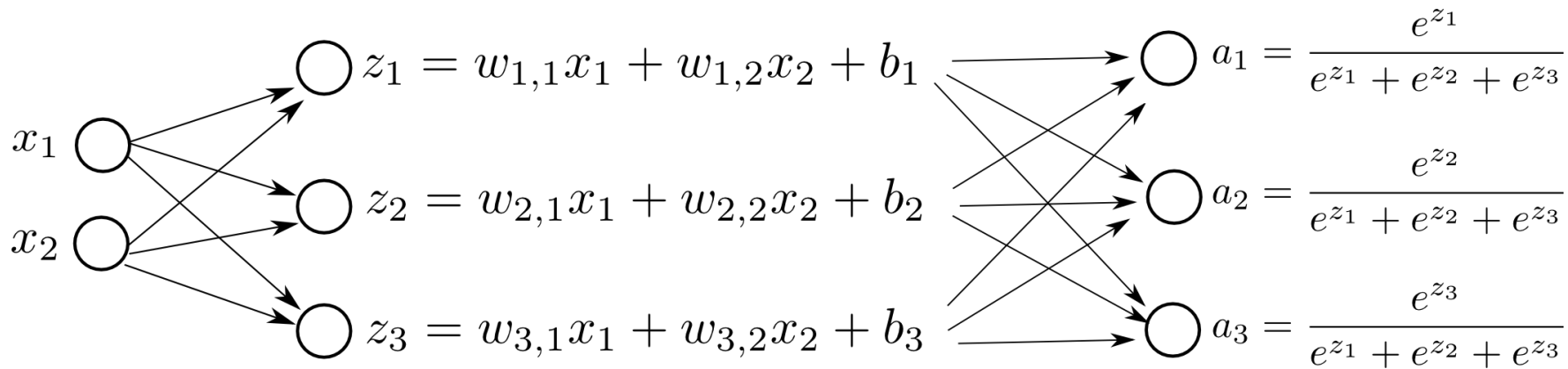
$$W = \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

For $x = (x_1, x_2)$

$$z = Wx + b$$

$$a = \text{softmax}(z)$$

Softmax Regression



```
W = np.array([[ 0.31, 3.95],
               [ 7.07, -0.23],
               [-6.27, -2.35]])
```

```
b = np.array([[ 1.2 ],
               [ 2.93 ],
               [-4.14 ]])
```

```
def forward(W, b, x):
    z = np.matmul(W, x) + b
    a = softmax(z)
```

```
    return z, a
```

```
# X[0,:] has the shape (1,2)
# reshape from (1,2) --> (2,1)
x = X[0,:].reshape(2,1)
z, a = forward(W, b, x)
```

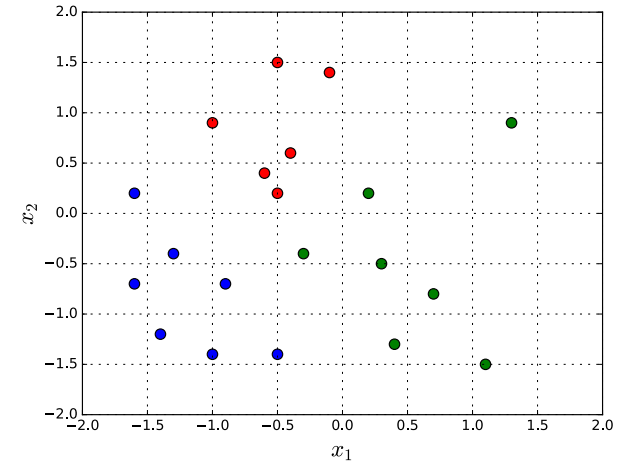
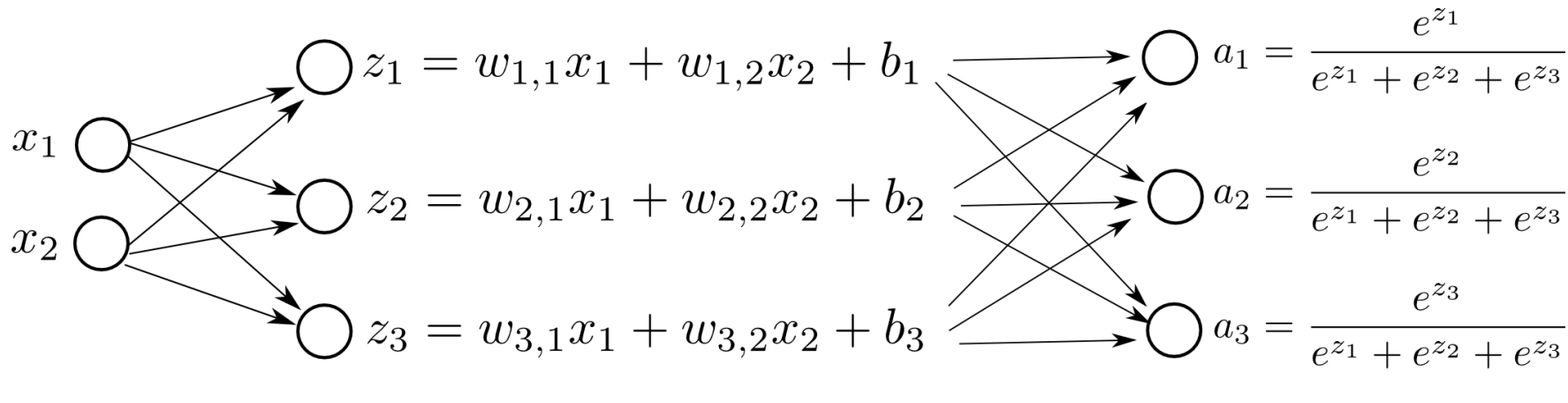
```
print(x.squeeze())
print(z.squeeze())
print(a.squeeze())
print(y[0])
```

```
[-0.1  1.4]
[ 6.699  1.901 -6.803]
[0.992  0.008  0. ]
0
```



LOSS FUNCTION & GRADIENT COMPUTATION

Loss Function



For example.

3 datapoints:

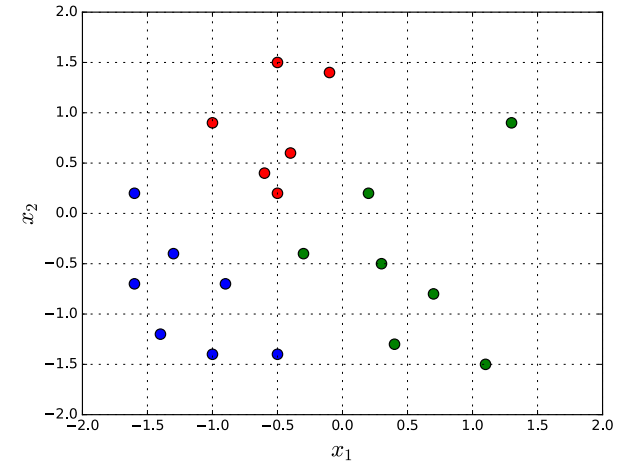
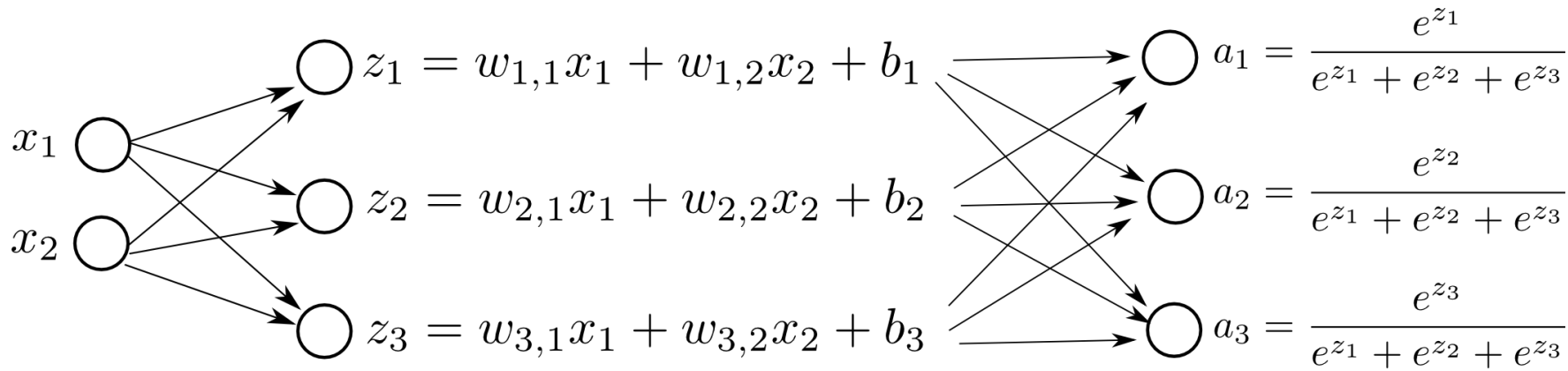
$$\mathbf{x}^{(1)} = \begin{bmatrix} -0.1 \\ 1.4 \end{bmatrix}, \quad y^{(1)} = 0, \quad \mathbf{x}^{(2)} = \begin{bmatrix} 1.3 \\ 0.9 \end{bmatrix}, \quad y^{(2)} = 1, \quad \mathbf{x}^{(3)} = \begin{bmatrix} -1.4 \\ -1.1 \end{bmatrix}, \quad y^{(3)} = 2$$

Suppose that – predicted (classifier):

$$\mathbf{y}^{(1)} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{a}^{(1)} = \begin{bmatrix} 0.9 \\ 0.1 \\ 0 \end{bmatrix}, \quad \mathbf{y}^{(2)} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{a}^{(2)} = \begin{bmatrix} 0.1 \\ 0.8 \\ 0.1 \end{bmatrix}, \quad \mathbf{y}^{(3)} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{a}^{(3)} = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.7 \end{bmatrix}$$

Good or not?

Loss Function



The likelihood that the model with parameter W, b predicts $\mathbf{a}^{(i)}$ fit with the label one-hot vector $\mathbf{y}^{(i)}$ of the i^{th} datapoint:

$$\prod_{j=1}^3 (a_j^{(i)})^{y_j^{(i)}}$$

For example: $\mathbf{y}^{(i)} = (1, 0, 0), \mathbf{a}^{(i)} = (0.9, 0.1, 0.0)$

$$\prod_{j=1}^3 (a_j^{(i)})^{y_j^{(i)}} = 0.9^1 \times 0.1^0 \times 0.0^0 = 0.9$$



MLE – Cross Entropy Loss

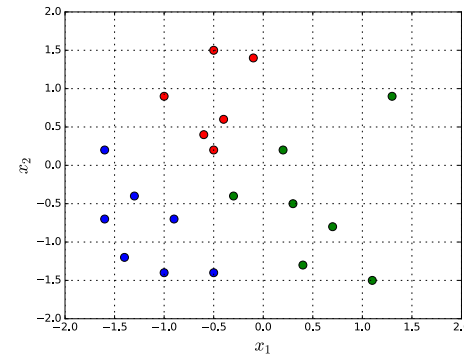
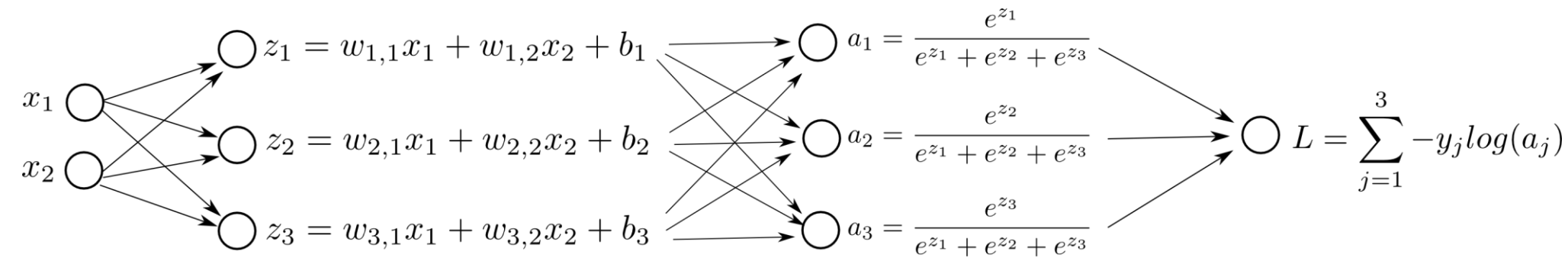
$$\begin{aligned}\hat{W}, \hat{\mathbf{b}} &= \arg \max_{W, \mathbf{b}} \prod_{i=1}^N \prod_{j=1}^3 \left(a_j^{(i)}\right)^{y_j^{(i)}} \text{ wrt. } \mathbf{a}^{(i)} = \text{softmax}(W\mathbf{x}^{(i)} + \mathbf{b}) \\&= \arg \max_{W, \mathbf{b}} \log \left(\prod_{i=1}^N \prod_{j=1}^3 \left(a_j^{(i)}\right)^{y_j^{(i)}} \right) = \arg \max_{W, \mathbf{b}} \sum_{i=1}^N \sum_{j=1}^3 \log \left(a_j^{(i)}\right)^{y_j^{(i)}} = \arg \max_{W, \mathbf{b}} \sum_{i=1}^N \sum_{j=1}^3 y_j^{(i)} \log \left(a_j^{(i)}\right) \\&= \arg \min_{W, \mathbf{b}} - \sum_{i=1}^N \sum_{j=1}^3 y_j^{(i)} \log \left(a_j^{(i)}\right) = \arg \min_{W, \mathbf{b}} \sum_{i=1}^N L(W, \mathbf{b}, \mathbf{x}^{(i)}, \mathbf{y}^{(i)}) = \arg \min_{W, \mathbf{b}} J(W, \mathbf{b})\end{aligned}$$

- Cross Entropy (CE) Loss

Cross Entropy Loss

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

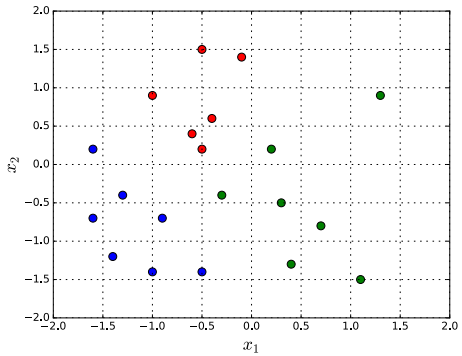
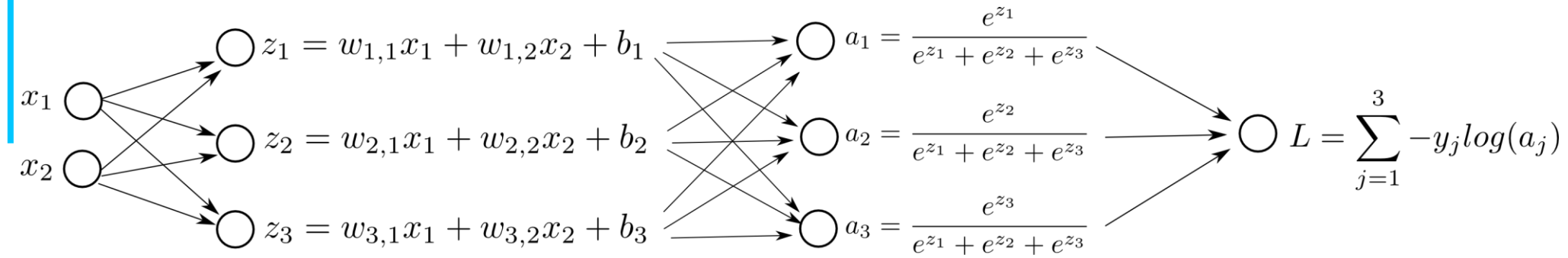
$$\mathbf{a} = \text{softmax}(\mathbf{z})$$



Cross Entropy (CE) Loss

$$J = - \sum_{i=1}^N \sum_{j=1}^C y_j^{(i)} \log(a_j^{(i)})$$

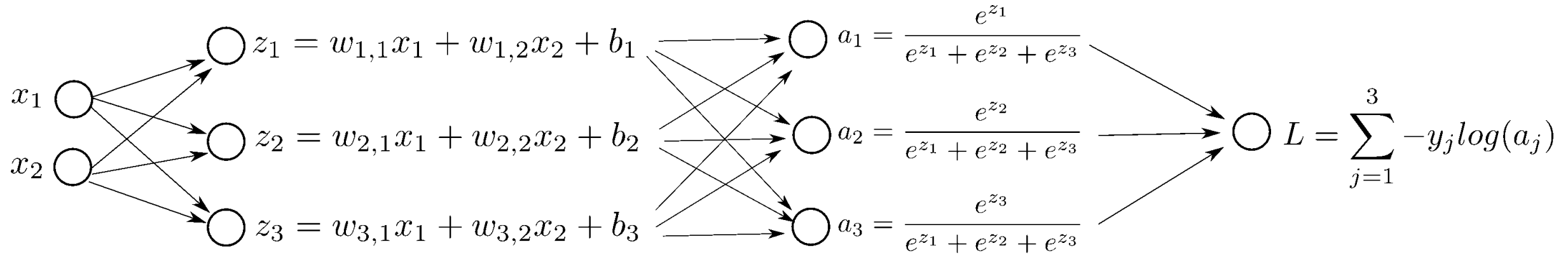
Gradient Descent



$$W = W - \alpha \frac{\partial L}{\partial W}$$

$$b = b - \alpha \frac{\partial L}{\partial b}$$

Gradient Computing

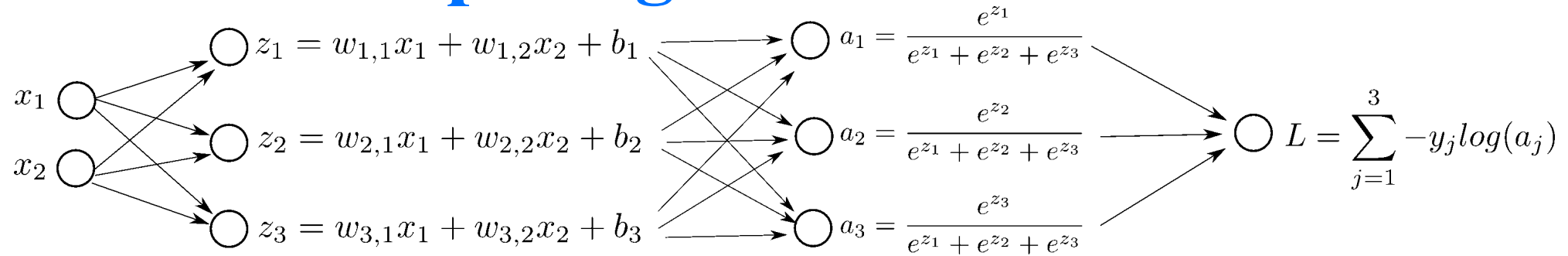


$$\frac{\partial L}{\partial w_{2,1}} = \sum_{i=1}^3 \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_2} \frac{\partial z_2}{\partial w_{2,1}}$$

$$\frac{\partial L}{\partial w_{m,n}} = \sum_{i=1}^3 \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_m} \frac{\partial z_m}{\partial w_{m,n}}, \quad \frac{\partial L}{\partial b_m} = \sum_{i=1}^3 \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_m} \frac{\partial z_m}{\partial b_m}$$



Gradient Computing



$$\frac{\partial L}{\partial w_{m,n}} = \sum_{i=1}^3 \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_m} \frac{\partial z_m}{\partial w_{m,n}}, \quad \frac{\partial L}{\partial b_m} = \sum_{i=1}^3 \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_m} \frac{\partial z_m}{\partial b_m}$$

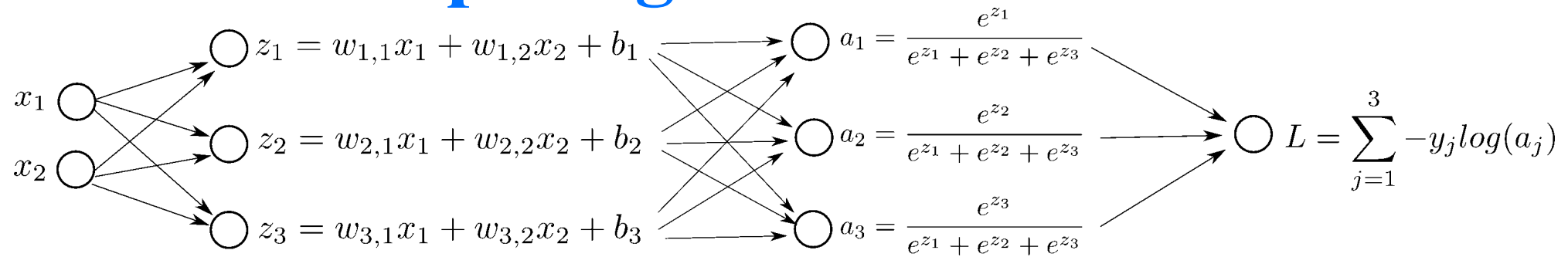
$$\frac{\partial z_m}{\partial w_{m,n}} = \frac{\partial}{\partial w_{m,n}} (w_{m,1}x_1 + w_{m,2}x_2 + \dots + w_{m,n}x_n + \dots + w_{m,d}x_d + b_m) = x_n$$

$$\frac{\partial z_m}{\partial b_m} = \frac{\partial}{\partial b_m} (w_{m,1}x_1 + w_{m,2}x_2 + \dots + w_{m,n}x_n + \dots + w_{m,d}x_d + b_m) = 1$$

$$\begin{array}{ccc} \frac{\partial z_1}{\partial w_{1,1}} = x_1, & \frac{\partial z_1}{\partial w_{1,2}} = x_2, & \frac{\partial z_1}{\partial b_1} = 1 \\ \frac{\partial z_2}{\partial w_{2,1}} = x_1, & \frac{\partial z_2}{\partial w_{2,2}} = x_2, & \frac{\partial z_2}{\partial b_2} = 1 \\ \frac{\partial z_3}{\partial w_{3,1}} = x_1, & \frac{\partial z_3}{\partial w_{3,2}} = x_2, & \frac{\partial z_3}{\partial b_3} = 1 \end{array}$$



Gradient Computing



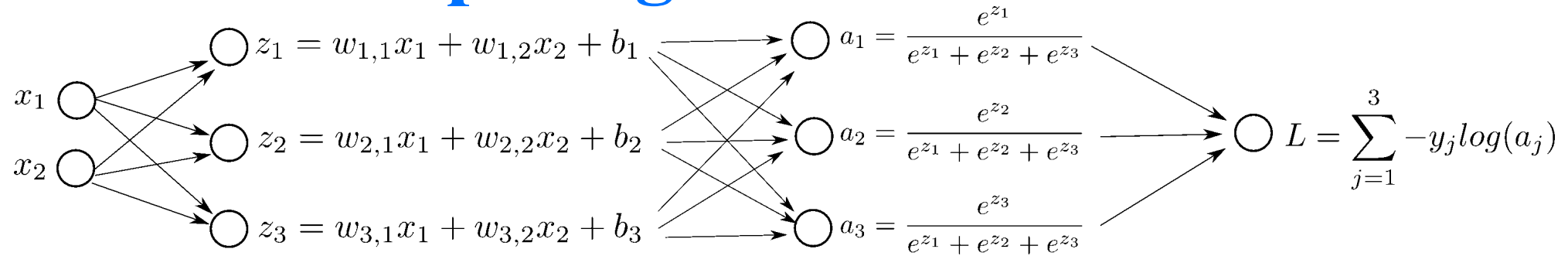
$$\frac{\partial L}{\partial w_{m,n}} = \sum_{i=1}^3 \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_m} \frac{\partial z_m}{\partial w_{m,n}}, \quad \frac{\partial L}{\partial b_m} = \sum_{i=1}^3 \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_m} \frac{\partial z_m}{\partial b_m}$$

$$\frac{\partial L}{\partial a_i} = \frac{\partial}{\partial a_i} \left(-\sum_{j=1}^c y_j \log(a_j) \right) = \frac{\partial}{\partial a_i} (-y_i \log(a_i)) = \frac{-y_i}{a_i}$$

$$\frac{\partial L}{\partial \mathbf{a}} = \begin{bmatrix} \frac{\partial L}{\partial a_1} \\ \frac{\partial L}{\partial a_2} \\ \frac{\partial L}{\partial a_3} \end{bmatrix} = \begin{bmatrix} \frac{-y_1}{a_1} \\ \frac{-y_2}{a_2} \\ \frac{-y_3}{a_3} \end{bmatrix}$$



Gradient Computing



$$\frac{\partial L}{\partial w_{m,n}} = \sum_{i=1}^3 \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_m} \frac{\partial z_m}{\partial w_{m,n}}, \quad \frac{\partial L}{\partial b_m} = \sum_{i=1}^3 \frac{\partial L}{\partial a_i} \frac{\partial a_i}{\partial z_m} \frac{\partial z_m}{\partial b_m}$$

$$\frac{\partial a_i}{\partial z_m} = \frac{\partial}{\partial z_m} \left(\frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \right)$$

$$\frac{d}{dx} \frac{f(x)}{g(x)} = \frac{f'(x)g(x) - g'(x)f(x)}{(g(x))^2}$$



Softmax Derivative

➤ $i = m$

$$\begin{aligned}\frac{\partial a_i}{\partial z_m} &= \frac{\partial}{\partial z_m} \left(\frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \right) = \frac{(e^{z_i})' (\sum_{j=1}^C e^{z_j}) - (\sum_{j=1}^C e^{z_j})' e^{z_i}}{(\sum_{j=1}^C e^{z_j})^2} \\ &= \frac{e^{z_i} (\sum_{j=1}^C e^{z_j}) - (e^{z_m}) e^{z_i}}{(\sum_{j=1}^C e^{z_j})^2} \\ &= \frac{e^{z_i} (\sum_{j=1}^C e^{z_j} - e^{z_m})}{(\sum_{j=1}^C e^{z_j})^2} = \frac{e^{z_m} (\sum_{j=1}^C e^{z_j} - e^{z_m})}{(\sum_{j=1}^C e^{z_j})^2} \\ &= \frac{e^{z_m}}{\sum_{j=1}^C e^{z_j}} \frac{(\sum_{j=1}^C e^{z_j} - e^{z_m})}{\sum_{j=1}^C e^{z_j}} = \frac{e^{z_m}}{\sum_{j=1}^C e^{z_j}} \left(\frac{\sum_{j=1}^C e^{z_j}}{\sum_{j=1}^C e^{z_j}} - \frac{e^{z_m}}{\sum_{j=1}^C e^{z_j}} \right) \\ &= a_m (1 - a_m)\end{aligned}$$



Softmax Derivative

➤ $i \neq m$

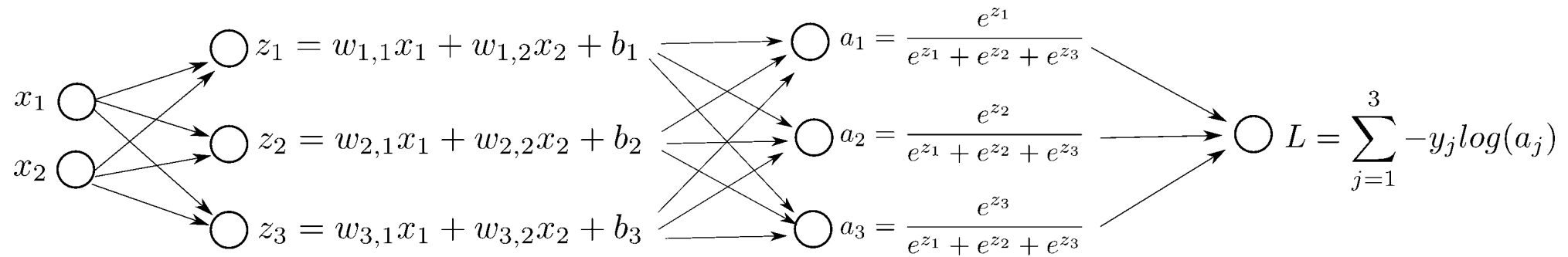
$$\begin{aligned}\frac{\partial a_i}{\partial z_m} &= \frac{\partial}{\partial z_m} \left(\frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \right) = \frac{(e^{z_i})' (\sum_{j=1}^C e^{z_j}) - (\sum_{j=1}^C e^{z_j})' e^{z_i}}{(\sum_{j=1}^C e^{z_j})^2} \\ &= \frac{0 - (e^{z_m}) e^{z_i}}{(\sum_{j=1}^C e^{z_j})^2} \\ &= \frac{-(e^{z_m}) e^{z_i}}{(\sum_{j=1}^C e^{z_j})^2} = \\ &= -\frac{e^{z_m}}{\sum_{j=1}^C e^{z_j}} \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \\ &= -a_m a_i\end{aligned}$$



Softmax Derivative

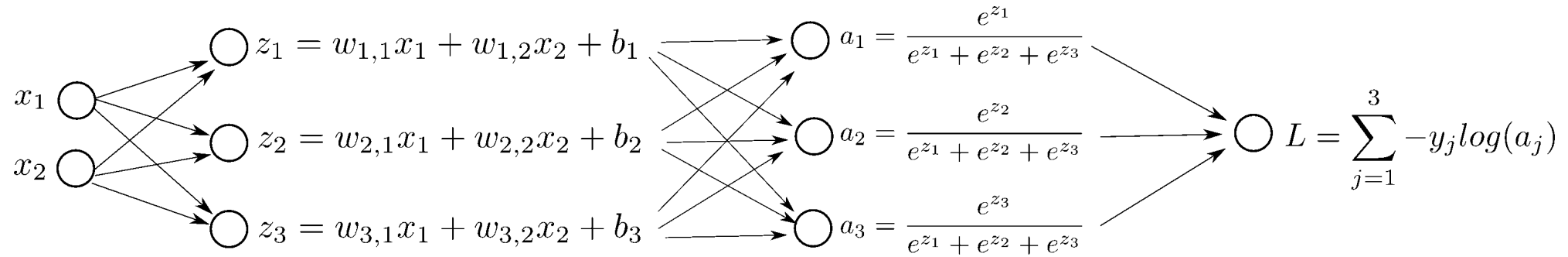
$$\frac{\partial a_i}{\partial z_m} = \begin{cases} a_m(1 - a_m), & \text{nếu } i = m \\ -a_m a_i, & \text{nếu } i \neq m \end{cases}$$

$$\begin{aligned} \frac{\partial \mathbf{a}}{\partial \mathbf{z}} &= \begin{bmatrix} \frac{\partial a_1}{\partial z_1} & \frac{\partial a_2}{\partial z_1} & \frac{\partial a_3}{\partial z_1} \\ \frac{\partial a_1}{\partial z_2} & \frac{\partial a_2}{\partial z_2} & \frac{\partial a_3}{\partial z_2} \\ \frac{\partial a_1}{\partial z_3} & \frac{\partial a_2}{\partial z_3} & \frac{\partial a_3}{\partial z_3} \end{bmatrix} = \begin{bmatrix} a_1(1 - a_1) & a_1(-a_2) & a_1(-a_3) \\ a_2(-a_1) & a_2(1 - a_2) & a_2(-a_3) \\ a_3(-a_1) & a_3(-a_2) & a_3(1 - a_3) \end{bmatrix} \\ &= \begin{bmatrix} a_1 & a_1 & a_1 \\ a_2 & a_2 & a_2 \\ a_3 & a_3 & a_3 \end{bmatrix} \circ \left(\begin{bmatrix} (1 - a_1) & (-a_2) & (-a_3) \\ (-a_1) & (1 - a_2) & 0 \\ (-a_1) & (-a_2) & (1 - a_3) \end{bmatrix} \right) \\ &= \begin{bmatrix} a_1 & a_1 & a_1 \\ a_2 & a_2 & a_2 \\ a_3 & a_3 & a_3 \end{bmatrix} \circ \left(\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} a_1 & a_2 & a_3 \\ a_1 & a_2 & a_3 \\ a_1 & a_2 & a_3 \end{bmatrix} \right) \\ &= \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} [1 \quad 1 \quad 1] \circ \left(\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} [a_1 \quad a_2 \quad a_3] \right) = \mathbf{a} \mathbf{1}^T \circ (\mathbf{I} - \mathbf{1} \mathbf{a}^T) \end{aligned}$$



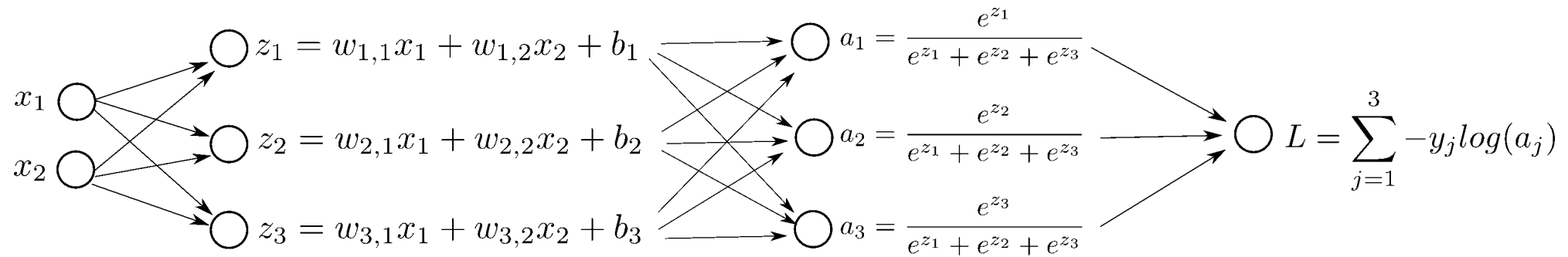
$$\frac{\partial L}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial L}{\partial w_{1,1}} & \frac{\partial L}{\partial w_{1,2}} \\ \frac{\partial L}{\partial w_{2,1}} & \frac{\partial L}{\partial w_{2,2}} \\ \frac{\partial L}{\partial w_{3,1}} & \frac{\partial L}{\partial w_{3,2}} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial w_{1,1}} & \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial w_{1,2}} \\ \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial w_{2,1}} & \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial w_{2,2}} \\ \frac{\partial L}{\partial z_3} \frac{\partial z_3}{\partial w_{3,1}} & \frac{\partial L}{\partial z_3} \frac{\partial z_3}{\partial w_{3,2}} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial z_1} x_1 & \frac{\partial L}{\partial z_1} x_2 \\ \frac{\partial L}{\partial z_2} x_1 & \frac{\partial L}{\partial z_2} x_2 \\ \frac{\partial L}{\partial z_3} x_1 & \frac{\partial L}{\partial z_3} x_2 \end{bmatrix} = \frac{\partial L}{\partial \mathbf{z}} \begin{bmatrix} x_1 & x_2 \end{bmatrix} = \frac{\partial L}{\partial \mathbf{z}} \mathbf{x}^T$$

$$\frac{\partial L}{\partial \mathbf{b}} = \begin{bmatrix} \frac{\partial L}{\partial b_1} \\ \frac{\partial L}{\partial b_2} \\ \frac{\partial L}{\partial b_3} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial b_1} \\ \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial b_2} \\ \frac{\partial L}{\partial z_3} \frac{\partial z_3}{\partial b_3} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial z_1} 1 \\ \frac{\partial L}{\partial z_2} 1 \\ \frac{\partial L}{\partial z_3} 1 \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial z_1} \\ \frac{\partial L}{\partial z_2} \\ \frac{\partial L}{\partial z_3} \end{bmatrix} = \frac{\partial L}{\partial \mathbf{z}}$$



$$\frac{\partial L}{\partial \mathbf{z}} = \begin{bmatrix} \frac{\partial L}{\partial z_1} \\ \frac{\partial L}{\partial z_2} \\ \frac{\partial L}{\partial z_3} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial a_1} \frac{\partial a_1}{\partial z_1} + \frac{\partial L}{\partial a_2} \frac{\partial a_2}{\partial z_1} + \frac{\partial L}{\partial a_3} \frac{\partial a_3}{\partial z_1} \\ \frac{\partial L}{\partial a_1} \frac{\partial a_1}{\partial z_2} + \frac{\partial L}{\partial a_2} \frac{\partial a_2}{\partial z_2} + \frac{\partial L}{\partial a_3} \frac{\partial a_3}{\partial z_2} \\ \frac{\partial L}{\partial a_1} \frac{\partial a_1}{\partial z_3} + \frac{\partial L}{\partial a_2} \frac{\partial a_2}{\partial z_3} + \frac{\partial L}{\partial a_3} \frac{\partial a_3}{\partial z_3} \end{bmatrix} = \begin{bmatrix} \frac{\partial a_1}{\partial z_1} & \frac{\partial a_2}{\partial z_1} & \frac{\partial a_3}{\partial z_1} \\ \frac{\partial a_1}{\partial z_2} & \frac{\partial a_2}{\partial z_2} & \frac{\partial a_3}{\partial z_2} \\ \frac{\partial a_1}{\partial z_3} & \frac{\partial a_2}{\partial z_3} & \frac{\partial a_3}{\partial z_3} \end{bmatrix} \begin{bmatrix} \frac{\partial L}{\partial a_1} \\ \frac{\partial L}{\partial a_2} \\ \frac{\partial L}{\partial a_3} \end{bmatrix} = \frac{\partial \mathbf{a}}{\partial \mathbf{z}} \frac{\partial L}{\partial \mathbf{a}}$$

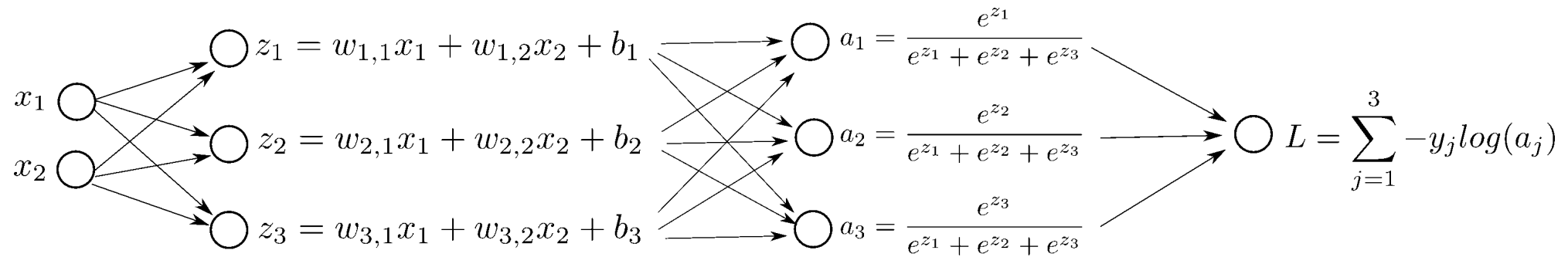
$$\frac{\partial \mathbf{a}}{\partial \mathbf{z}} = \begin{bmatrix} a_1(1-a_1) & a_1(-a_2) & a_1(-a_3) \\ a_2(-a_1) & a_2(1-a_2) & a_2(-a_3) \\ a_3(-a_1) & a_3(-a_2) & a_3(1-a_3) \end{bmatrix}, \quad \frac{\partial L}{\partial \mathbf{a}} = \begin{bmatrix} -\frac{y_1}{a_1} \\ -\frac{y_2}{a_2} \\ -\frac{y_3}{a_3} \end{bmatrix}$$



$$\frac{\partial L}{\partial \mathbf{z}} = \frac{\partial \mathbf{a}}{\partial \mathbf{z}} \frac{\partial L}{\partial \mathbf{a}} = \begin{bmatrix} a_1(1-a_1) & a_1(-a_2) & a_1(-a_3) \\ a_2(-a_1) & a_2(1-a_2) & a_2(-a_3) \\ a_3(-a_1) & a_3(-a_2) & a_3(1-a_3) \end{bmatrix} \begin{bmatrix} \frac{-y_1}{a_1} \\ \frac{-y_2}{a_2} \\ \frac{-y_3}{a_3} \end{bmatrix}$$

$$= \begin{bmatrix} -y_1(1-a_1) + y_2 a_1 + y_3 a_1 \\ y_1 a_2 - y_2(1-a_2) + y_3 a_2 \\ y_1 a_3 + y_2 a_3 - y_3(1-a_3) \end{bmatrix} = \begin{bmatrix} (a_1 - 1) & a_1 & a_1 \\ a_2 & (a_2 - 1) & a_2 \\ a_3 & a_3 & (a_3 - 1) \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

$$= \left(\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} [1 \quad 1 \quad 1] - \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = (\mathbf{a} \mathbf{1}^T - \mathbf{I}) \mathbf{y} = \mathbf{a} \mathbf{1}^T \mathbf{y} - \mathbf{I} \mathbf{y} = \mathbf{a} - \mathbf{y}$$



Gradient của Cross Entropy (CE) Loss của hồi quy softmax (xét trên 1 điểm dữ liệu):

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{z}} \begin{bmatrix} x_1 & x_2 \end{bmatrix} = \frac{\partial L}{\partial \mathbf{z}} \mathbf{x}^T = (\mathbf{a} - \mathbf{y}) \mathbf{x}^T$$

$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{z}} = (\mathbf{a} - \mathbf{y})$$

Gradient (xét trên cả tập dữ liệu):

$$\frac{\partial J}{\partial \mathbf{W}} = \sum_{i=1}^N (\mathbf{a}^{(i)} - \mathbf{y}^{(i)}) \mathbf{x}^{(i)T}, \quad \frac{\partial J}{\partial \mathbf{b}} = \sum_{i=1}^N (\mathbf{a}^{(i)} - \mathbf{y}^{(i)})$$



Gradient Descent

```
learning_rate = 2.0
```

```
num_epochs = 40
```

```
def compute_gradient(x, y, z, a):
```

```
    dz = a - y
```

```
    dW = np.matmul(dz, x.T)
```

```
    db = dz.copy()
```

```
    return dW, db
```

```
def gradient_descent(W, b, dW, db, learning_rate):
```

```
    W = W - learning_rate * dW
```

```
    b = b - learning_rate * db
```

```
    return W, b
```

```
W = np.random.rand(3,2)
```

```
b = np.zeros((3,1))
```

```
for i in range(num_epochs):
```

```
    dW = np.zeros(W.shape)
```

```
    db = np.zeros(b.shape)
```

```
    L = 0
```

```
    for j in range(X.shape[0]):
```

```
        x_j = X[j,:].reshape(2,1)
```

```
        y_j = Y[j,:].reshape(3,1)
```

```
        z_j, a_j = forward(W, b, x_j)
```

```
        loss_j = compute_loss(y_j, z_j)
```

```
        dW_j, db_j = compute_gradient(x_j, y_j, z_j, a_j)
```

```
        dW += dW_j
```

```
        db += db_j
```

```
        L += loss_j
```

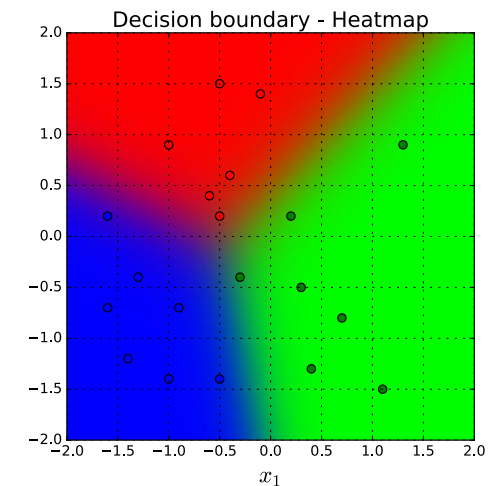
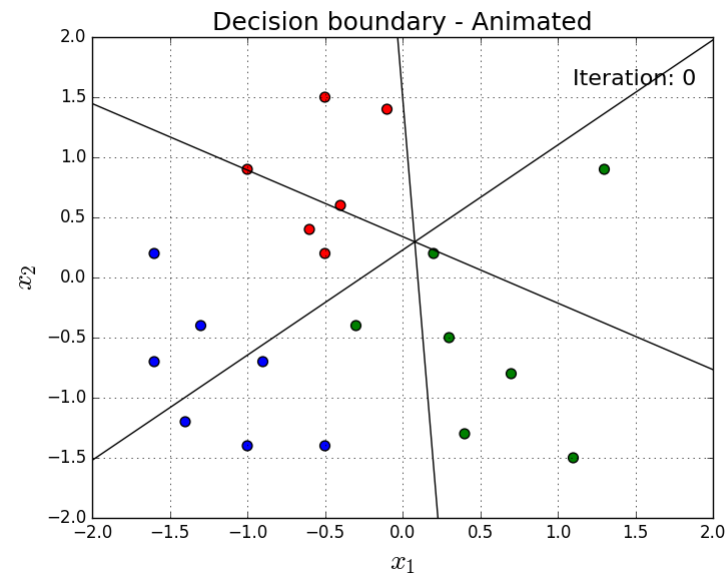
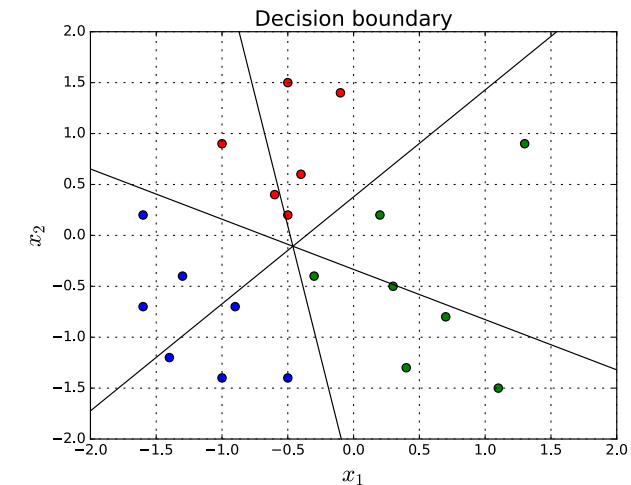
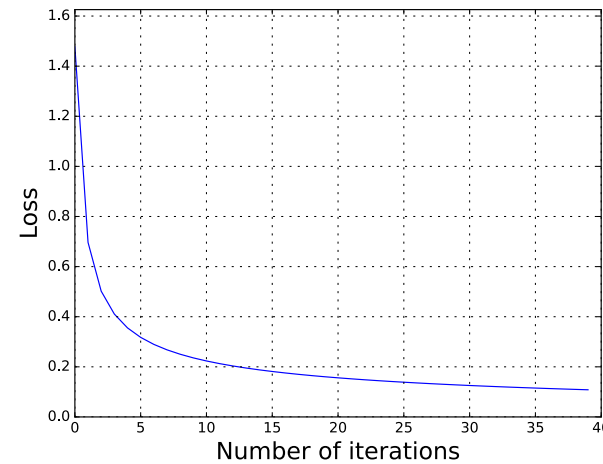
```
    dW = (1.0/X.shape[0]) * dW
```

```
    db = (1.0/X.shape[0]) * db
```

```
    L = (1.0/X.shape[0]) * L
```

```
    W, b = gradient_descent(W, b, dW, db, learning_rate)
```

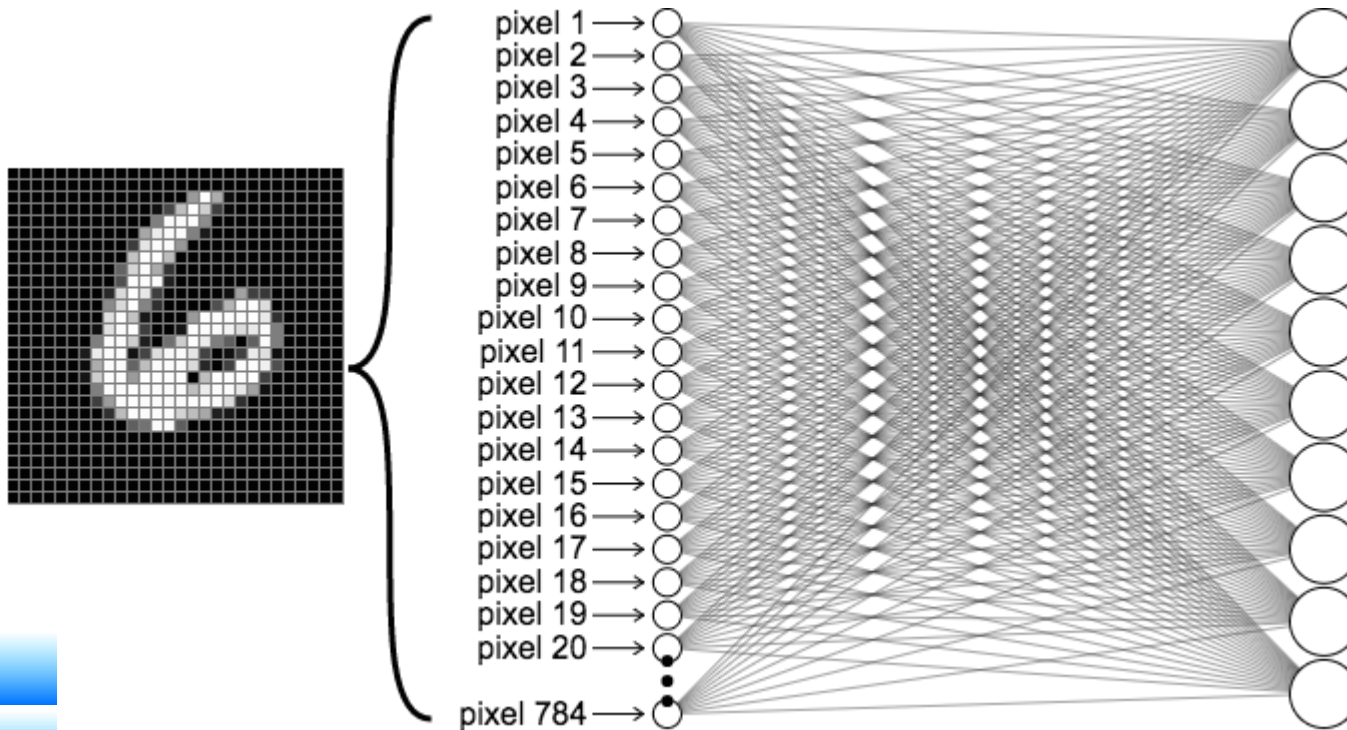
Thực hiện bởi Trường Đại học Công nghệ Thông tin, ĐHQG-HCM





IMPLEMENTATION & ANALYSIS

MNIST – Classification



```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader
import torchvision.datasets as datasets
import torchvision.transforms as transforms

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Create a Softmax Regression Model
class SoftmaxRegression(nn.Module):
    def __init__(self, input_size, num_classes): # digits 0, 1, 2, ..., 9
        super(SoftmaxRegression, self).__init__()
        self.layer = nn.Linear(in_features=input_size, out_features=num_classes)

    def forward(self, x):
        out = self.layer(x)
        # out = torch.softmax(out)
        return out

# Hyperparameters
input_size = 784 # 28x28 image
num_classes = 10 # 10 digits 0, 1, 2, ..., 9
learning_rate = 0.001
batch_size = 64
num_epochs = 5

model = SoftmaxRegression(input_size=input_size,
                           num_classes=num_classes).to(device)
criterion = nn.CrossEntropyLoss() # In PyTorch, Softmax + NLL
optimizer = optim.SGD(model.parameters(), lr=learning_rate)
```



MNIST – Classification

```
# Load the dataset to our Google Drive
train_dataset = datasets.MNIST(root='/content/drive/MyDrive/datasets/mnist',
                                train=True, transform=transforms.ToTensor(), download=True)
test_dataset = datasets.MNIST(root='/content/drive/MyDrive/datasets/mnist',
                                train=False, transform=transforms.ToTensor(), download=True)
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(dataset=test_dataset, batch_size=batch_size, shuffle=False)

# Train our Softmax Regression model
for epoch in range(num_epochs):
    for batch_idx, (data, targets) in enumerate(train_loader):
        data = data.to(device) # Put images to the GPU if GPU is available
        targets = targets.to(device) # Put labels to the GPU if available
        # (64, 1, 28, 28) ----> (64, 784)
        data = data.reshape(data.shape[0], -1)

        # forward pass
        scores = model(data) # Pass one batch of training examples.
        loss = criterion(scores, targets) # Compute the loss function value J

        # backward pass
        optimizer.zero_grad() # Empty the memory
        loss.backward() # Compute the gradient dJ/dw
        optimizer.step() # One step of gradient descent

    if (batch_idx+1) % 100 == 0:
        print(f'Epoch {epoch+1}/{num_epochs}, Batch {batch_idx+1}, Loss {loss.item():.2f}')
```

```
Epoch 1/5, Batch 100, Loss 2.25
Epoch 1/5, Batch 200, Loss 2.10
Epoch 1/5, Batch 300, Loss 2.00
Epoch 1/5, Batch 400, Loss 1.93
Epoch 1/5, Batch 500, Loss 1.85
Epoch 1/5, Batch 600, Loss 1.87
Epoch 1/5, Batch 700, Loss 1.72
Epoch 1/5, Batch 800, Loss 1.64
Epoch 1/5, Batch 900, Loss 1.61
Epoch 2/5, Batch 100, Loss 1.53
.....
Epoch 5/5, Batch 100, Loss 0.97
Epoch 5/5, Batch 200, Loss 0.89
Epoch 5/5, Batch 300, Loss 0.96
Epoch 5/5, Batch 400, Loss 0.90
Epoch 5/5, Batch 500, Loss 0.86
Epoch 5/5, Batch 600, Loss 0.70
Epoch 5/5, Batch 700, Loss 0.89
Epoch 5/5, Batch 800, Loss 0.80
Epoch 5/5, Batch 900, Loss 0.73
```

MNIST – Classification

```
# Evaluate the accuracy of the trained model
def check_accuracy(loader, model):
    num_corrects = 0
    num_samples = 0

    model.eval() # Put model into evaluation mode
    with torch.no_grad():
        for data, targets in loader:
            data = data.to(device)
            targets = targets.to(device)
            data = data.reshape(data.shape[0], -1)

            # forward
            scores = model(data) # Scores of a batch ---> in the form of (64, 10)
            #image 1: [s0, s1, s2, ..., s9] ---> s2, 2
            #image 2: [s0, s1, s2, ..., s9] ---> s10, 10
            #.....
            #image 64:[s0, s1, s2, ..., s9] ---> s7, 7
            _, preds = scores.max(1)

            num_corrects += (preds == targets).sum()
            num_samples += preds.size(0)

        print(f'We get {num_corrects}/{num_samples} correct. Accuracy =
              {float(num_corrects)/float(num_samples)*100.0:.2f}')

check_accuracy(test_loader, model)

We get 8477/10000 correct. Accuracy = 84.77
```

```
import matplotlib.pyplot as plt
image, label = test_dataset[1802]
plt.imshow(image.squeeze(), cmap='gray')

0 5 10 15 20 25
5 10 15 20 25
0 5 10 15 20 25

image = image.to(device) # Put the image to the GPU if available
image = image.reshape(image.shape[0], -1) # Reshape the tensor

score = model(image)
print(score)
_, preds = score.max(1)
print(f'Our prediction is {preds.item()}')
tensor([[ -0.6124, -1.3164,  2.3769, -1.1956, -0.4008, -1.1456,
  0.4172,  0.2381,  1.4189, -0.6130]], device='cuda:0',
grad_fn=<AddmmBackward0>)
Our prediction is 2

torch.nn.functional.softmax(score, dim=1)
tensor([[ 0.0267,  0.0132,  0.5298,  0.0149,  0.0329,  0.0156,
  0.0746,  0.0624,  0.2032,  0.0266]], device='cuda:0',
grad_fn=<SoftmaxBackward0>)
```

MNIST – Classification

```
# Evaluate the accuracy of the trained model
def check_accuracy(loader, model):
    num_corrects = 0
    num_samples = 0

    model.eval() # Put model into evaluation mode
    with torch.no_grad():
        for data, targets in loader:
            data = data.to(device)
            targets = targets.to(device)
            data = data.reshape(data.shape[0], -1)

            # forward
            scores = model(data) # Scores of a batch ---> (64, 10)
            #image 1: [s0, s1, s2, ..., s9] ---> s2, 2
            #image 2: [s0, s1, s2, ..., s9] ---> s10, 10
            #.....
            #image 64:[s0, s1, s2, ..., s9] ---> s7, 7
            _, preds = scores.max(1)

            num_corrects += (preds == targets).sum()
            num_samples += preds.size(0)

        print(f'We get {num_corrects}/{num_samples} correct. Accuracy =
              {float(num_corrects)/float(num_samples)*100.0:.2f}')

check_accuracy(test_loader, model)

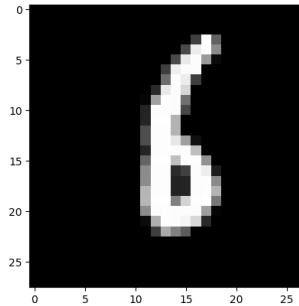
We get 8477/10000 correct. Accuracy = 84.77
```

```
import matplotlib.pyplot as plt
image, label = test_dataset[1100]
plt.imshow(image.squeeze(), cmap='gray')

image = image.to(device) # Put the image to the GPU if available
image = image.reshape(image.shape[0], -1) # Reshape the tensor

score = model(image)
print(score)
_, preds = score.max(1)
print(f'Our prediction is {preds.item()}')
tensor([[ -1.9898,  1.4574,  0.1530, -0.2754, -0.8373,  0.2092,
  1.7860, -0.9584,  0.6460, -0.1126]], device='cuda:0',
grad_fn=<AddmmBackward0>)
Our prediction is 6

torch.nn.functional.softmax(score, dim=1)
tensor([[ 0.0080,  0.2501,  0.0679,  0.0442,  0.0252,  0.0718,
  0.3474,  0.0223,  0.1111,  0.0520]], device='cuda:0',
grad_fn=<SoftmaxBackward0>)
```



MNIST – Classification

```
# Evaluate the accuracy of the trained model
def check_accuracy(loader, model):
    num_corrects = 0
    num_samples = 0

    model.eval() # Put model into evaluation mode
    with torch.no_grad():
        for data, targets in loader:
            data = data.to(device)
            targets = targets.to(device)
            data = data.reshape(data.shape[0], -1)

            # forward
            scores = model(data) # Scores of a batch ---> (64, 10)
            #image 1: [s0, s1, s2, ..., s9] ---> s2, 2
            #image 2: [s0, s1, s2, ..., s9] ---> s10, 10
            #.....
            #image 64:[s0, s1, s2, ..., s9] ---> s7, 7
            _, preds = scores.max(1)

            num_corrects += (preds == targets).sum()
            num_samples += preds.size(0)

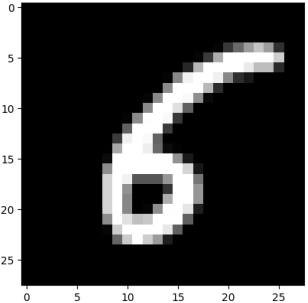
        print(f'We get {num_corrects}/{num_samples} correct. Accuracy =
              {float(num_corrects)/float(num_samples)*100.0:.2f}')

check_accuracy(test_loader, model)

We get 8477/10000 correct. Accuracy = 84.77
```

```
import matplotlib.pyplot as plt
image, label = test_dataset[217]
plt.imshow(image.squeeze(), cmap='gray')

0 5 10 15 20 25
5 10 15 20 25
0 5 10 15 20 25
```



```
image = image.to(device) # Put the image to the GPU if available
image = image.reshape(image.shape[0], -1) # Reshape the tensor

score = model(image)
print(score)
_, preds = score.max(1)
print(f'Our prediction is {preds.item()}')
tensor([[ 0.0747,  0.5930, -0.2984, -0.2330, -0.0943,  1.5958,
          0.3566, -1.4400,  0.8870, -0.9277]], device='cuda:0',
        grad_fn=<AddmmBackward0>)
Our prediction is 5

torch.nn.functional.softmax(score, dim=1)
tensor([[0.0730, 0.1227, 0.0503, 0.0537, 0.0617, 0.3343,
          0.0968, 0.0161, 0.1646, 0.0268]], device='cuda:0',
        grad_fn=<SoftmaxBackward0>)
```

Softmax regression

```
model.state_dict()
```

```
OrderedDict([
```

```
    ('layer.weight',
```

```
        tensor([[ 0.0349, -0.0092, 0.0314, ..., 0.0069, 0.0223, -0.0041],
```

```
              [ 0.0033, 0.0243, -0.0173, ..., -0.0064, -0.0092, 0.0318],
```

```
              [ 0.0033, -0.0335, 0.0165, ..., 0.0218, -0.0354, 0.0228],
```

```
              ...,
```

```
              [ 0.0270, -0.0133, 0.0106, ..., 0.0226, -0.0202, -0.0300],
```

```
              [-0.0277, 0.0045, 0.0304, ..., 0.0312, -0.0137, -0.0322],
```

```
              [-0.0027, 0.0342, 0.0262, ..., -0.0110, 0.0317, 0.0154]],
```

```
        device='cuda:0')),
```

```
    ('layer.bias',
```

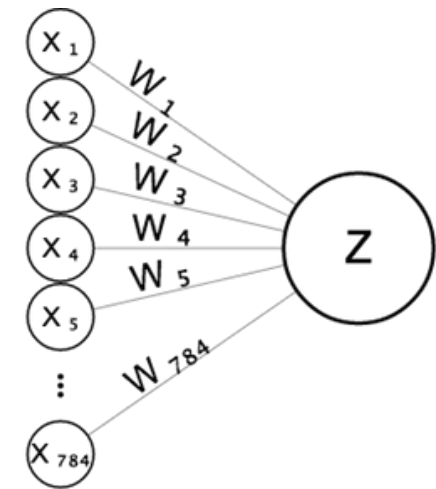
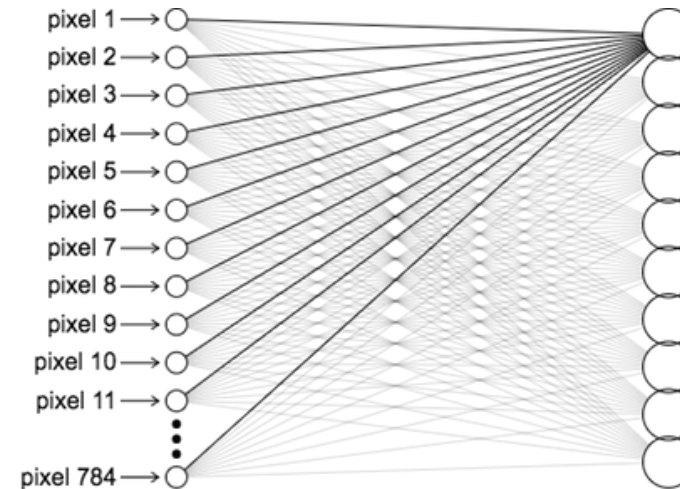
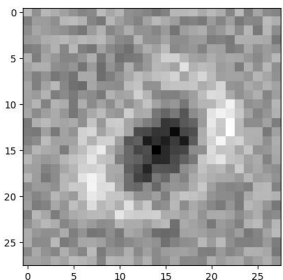
```
        tensor([-0.0714, 0.0465, -0.0165, 0.0134, -0.0157, 0.0391, -
```

```
0.0322, 0.0303, -0.0566, -0.0136], device='cuda:0'))])
```

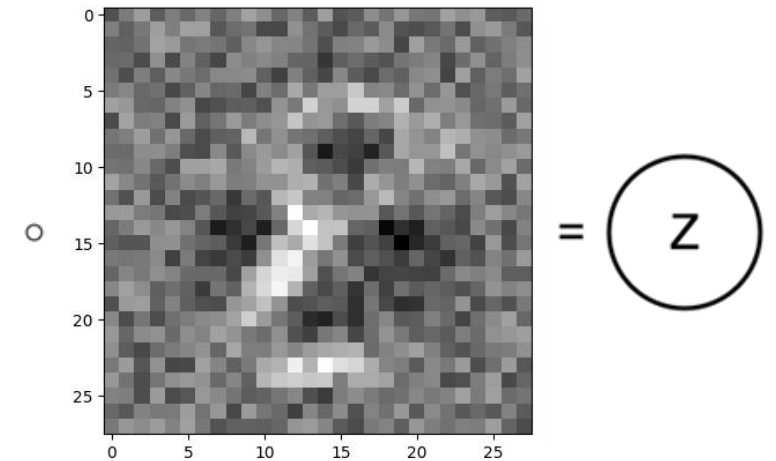
```
weight_digit_0 = model.state_dict()['layer.weight'][0]
```

```
weight_matrix = weight_digit_0.reshape(28, 28).cpu().numpy()
```

```
plt.imshow(weight_matrix, cmap='gray')
```



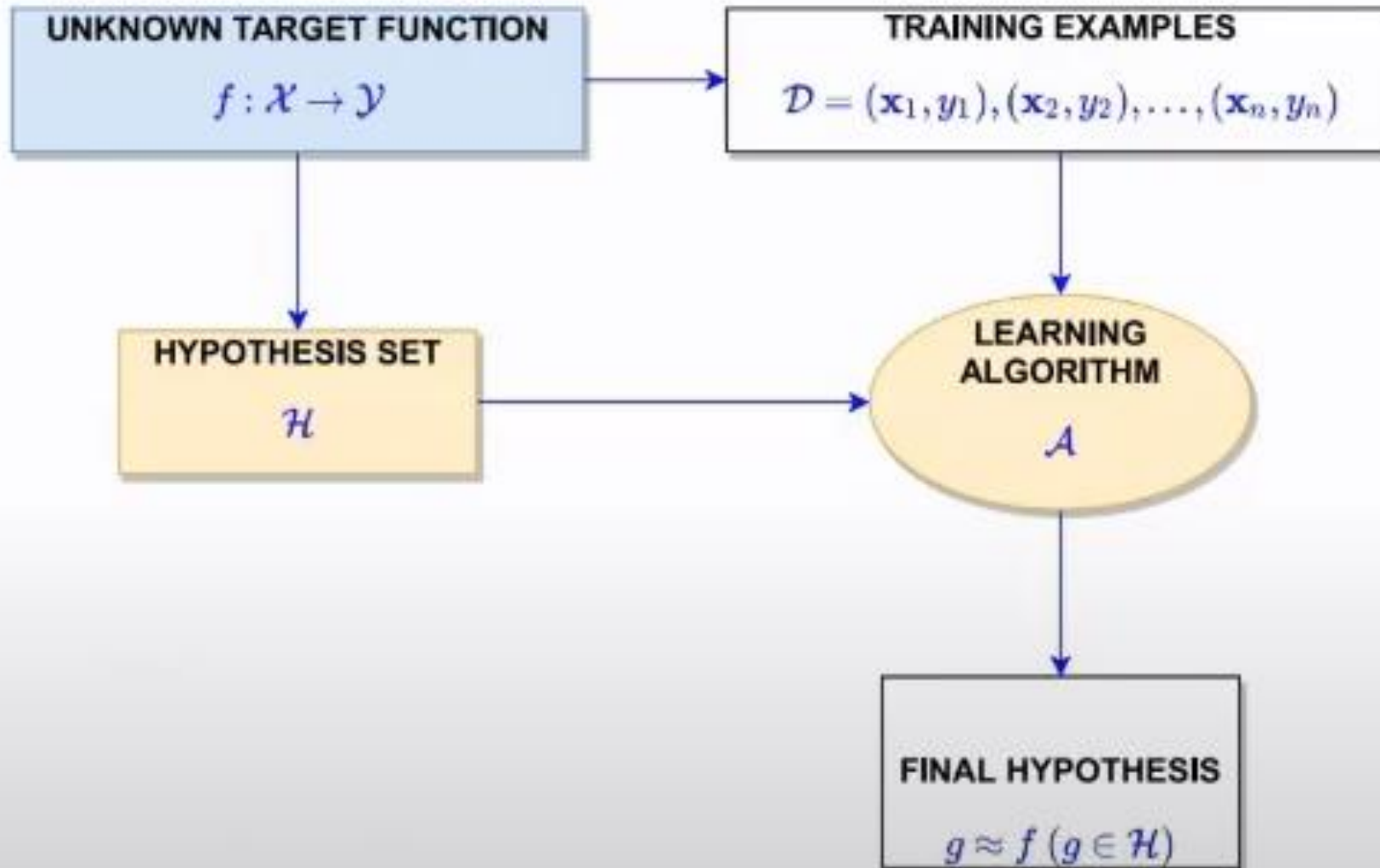
X_1	X_2	X_3	$\dots$	X_{28}
X_{29}	X_{30}	X_{31}		X_{56}
X_{57}	X_{58}	X_{59}		X_{84}
$\vdots$			$\ddots$	
X_{757}	X_{758}	X_{759}		X_{784}

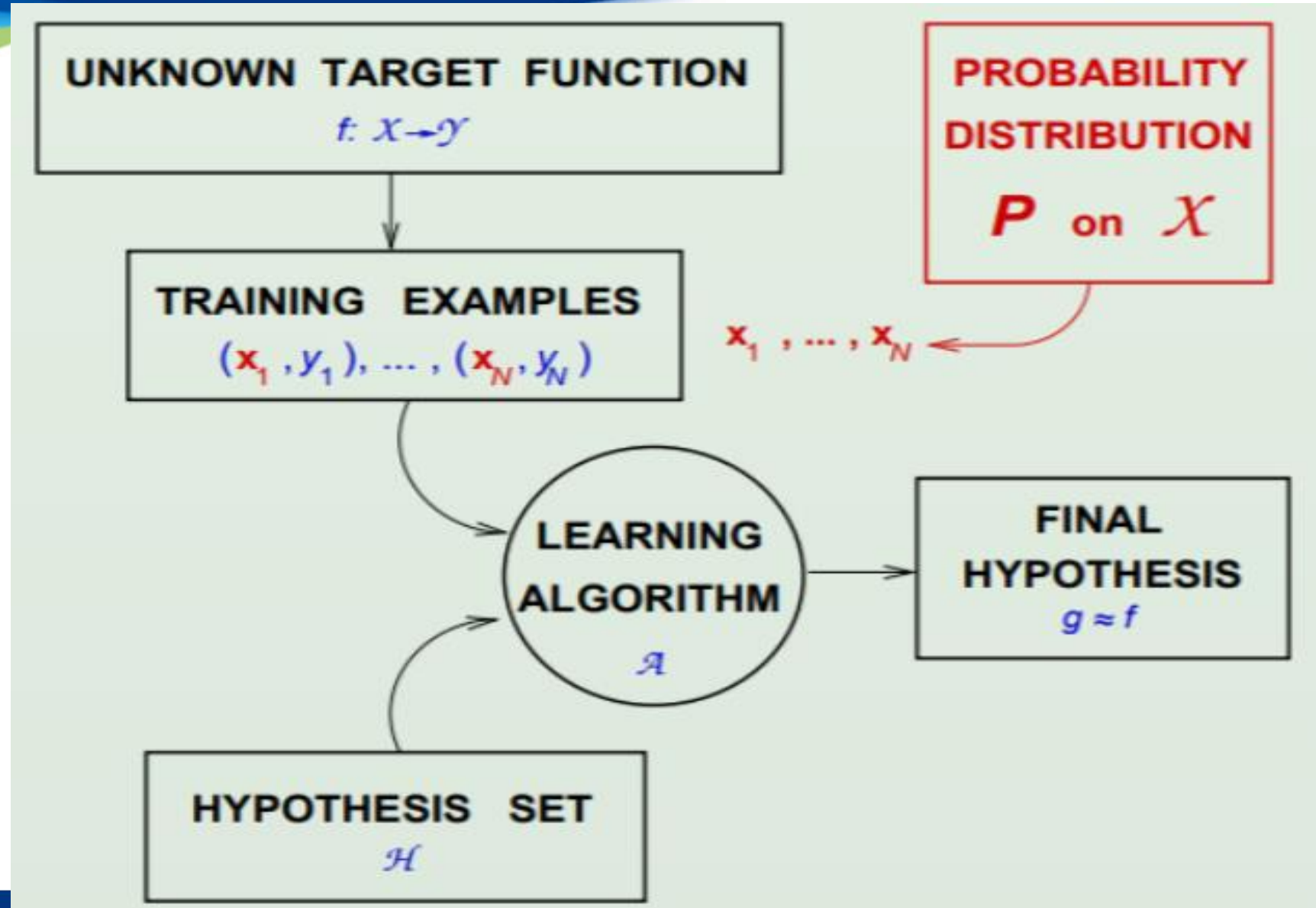


Fitting probability models

- Three major components of a machine learning system
 1. Data: $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n), \dots, (\mathbf{x}_N, y_N)\}$
 2. Models: deterministic functions or probabilistic models
 3. Learning: Training, and prediction/inference
- Good machine learning models: Perform well for unseen (untrained) data
- Machine learning algorithm: training and prediction

Learning Components





Models as Probabilistic Models: Parameter Estimation (MLE, MAP, Bayesian approach)

Fitting probability distributions

- Maximum Likelihood Estimation (ML Estimation or MLE)
- Maximum A Posteriori Estimation (MAP Estimation or MAP)
- Bayesian approach

$$\underbrace{p(\theta \mid D)}_{\text{posterior}} = \frac{\overbrace{p(D \mid \theta)}^{\text{likelihood}} \overbrace{p(\theta)}^{\text{prior}}}{\underbrace{p(D)}_{\text{evidence}}}$$

Maximum Likelihood Estimation

As the name suggests we find the parameters under which the data $\mathbf{x}_{1...I}$ is most likely.

$$\begin{aligned}\hat{\theta} &= \operatorname{argmax}_{\theta} [Pr(\mathbf{x}_{1...I}|\theta)] \\ &= \operatorname{argmax}_{\theta} \left[\prod_{i=1}^I Pr(\mathbf{x}_i|\theta) \right]\end{aligned}$$

We have assumed that data was **independent** (hence product)

$$p(x, y) = p(x)p(y).$$

Idea: define a function of the parameters called **likelihood function**. $p(x, y|z) = p(x|z)p(y|z)$

Maximum Likelihood Estimation

As the name suggests we find the parameters under which the data $\mathbf{x}_{1...I}$ is most likely.

$$\begin{aligned}\hat{\theta} &= \operatorname{argmax}_{\theta} [Pr(\mathbf{x}_{1...I}|\theta)] \\ &= \operatorname{argmax}_{\theta} \left[\prod_{i=1}^I Pr(\mathbf{x}_i|\theta) \right]\end{aligned}$$

We have assumed that data was **independent** (hence product)

Predictive Density:

Evaluate new data point $\mathbf{x}^*$ under probability distribution $Pr(\mathbf{x}^*|\hat{\theta})$
with best parameters

Fitting a normal distribution: MLE

As the name suggests we find the parameters under which the data $\mathbf{x}_{1...I}$ is most likely.

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} [Pr(\mathbf{x}_{1...I}|\boldsymbol{\theta})] \\ &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \left[\prod_{i=1}^I Pr(\mathbf{x}_i|\boldsymbol{\theta}) \right]\end{aligned}$$

Likelihood given by pdf

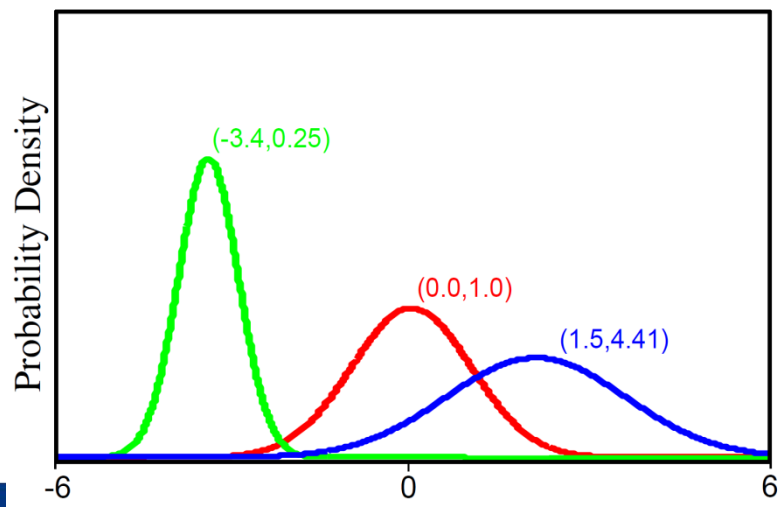
$$Pr(x|\mu, \sigma^2) = \operatorname{Norm}_x[\mu, \sigma^2] = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-0.5 \frac{(x - \mu)^2}{\sigma^2} \right]$$

Univariate Normal Distribution

$$Pr(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-0.5(x - \mu)^2 / \sigma^2 \right]$$

For short we write:

$$Pr(x) = \text{Norm}_x[\mu, \sigma^2]$$



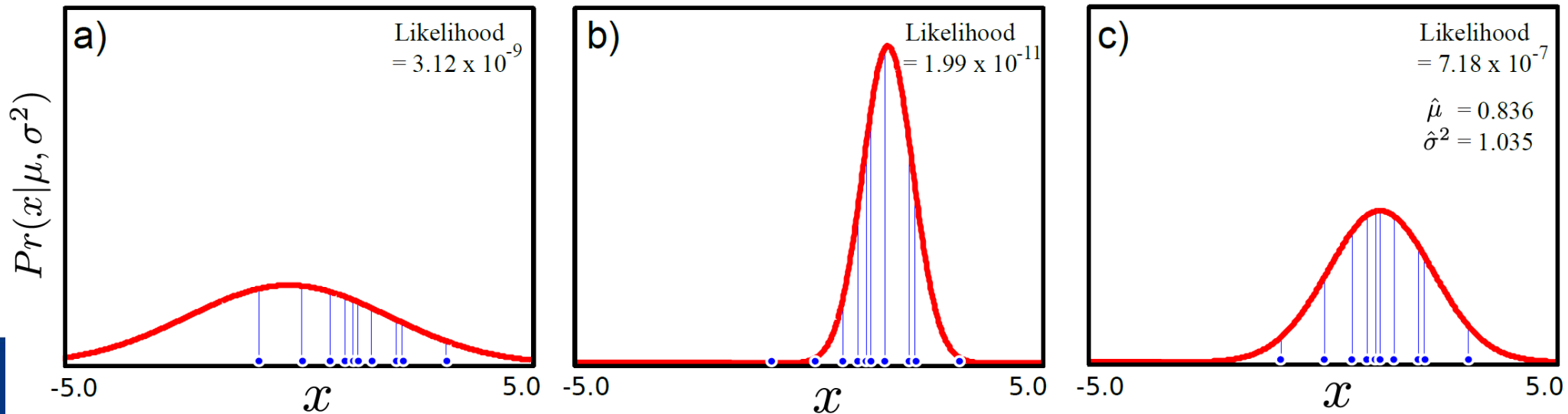
Univariate normal distribution describes single continuous variable.

Takes 2 parameters μ and $\sigma^2 > 0$

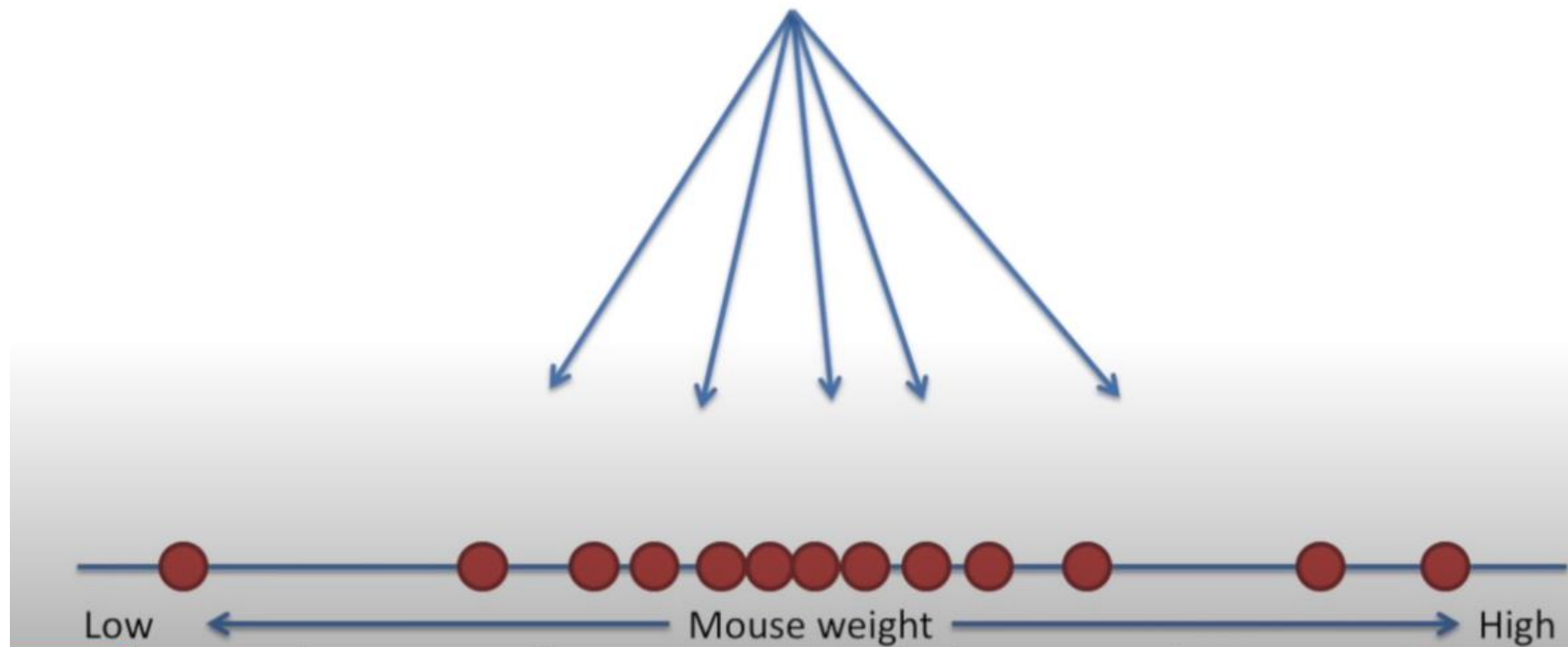
Fitting a normal distribution: MLE

$$\begin{aligned}
 Pr(x_{1...I}|\mu, \sigma^2) &= \prod_{i=1}^I Pr(x_i|\mu, \sigma^2) \\
 &= \prod_{i=1}^I \text{Norm}_{x_i}[\mu, \sigma^2] \\
 &= \frac{1}{(2\pi\sigma^2)^{I/2}} \exp \left[-0.5 \sum_{i=1}^I \frac{(x_i - \mu)^2}{\sigma^2} \right]
 \end{aligned}$$

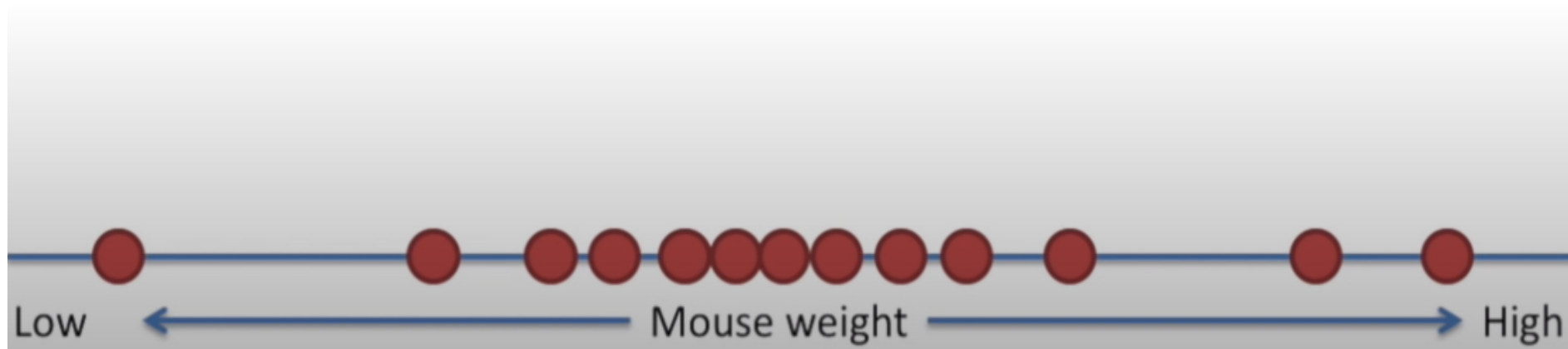
c) Better explains the data



Let's say we weighed a bunch of mice...

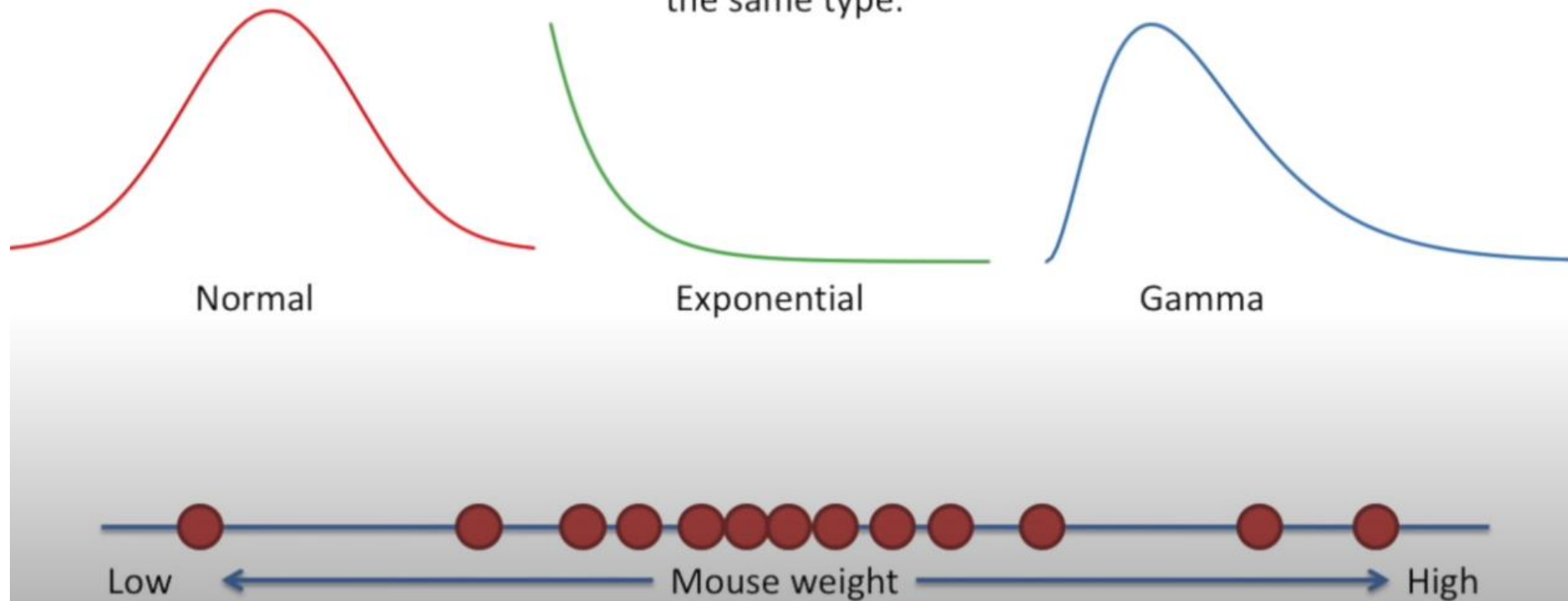


There are lots of different types of distributions
for different types of data...



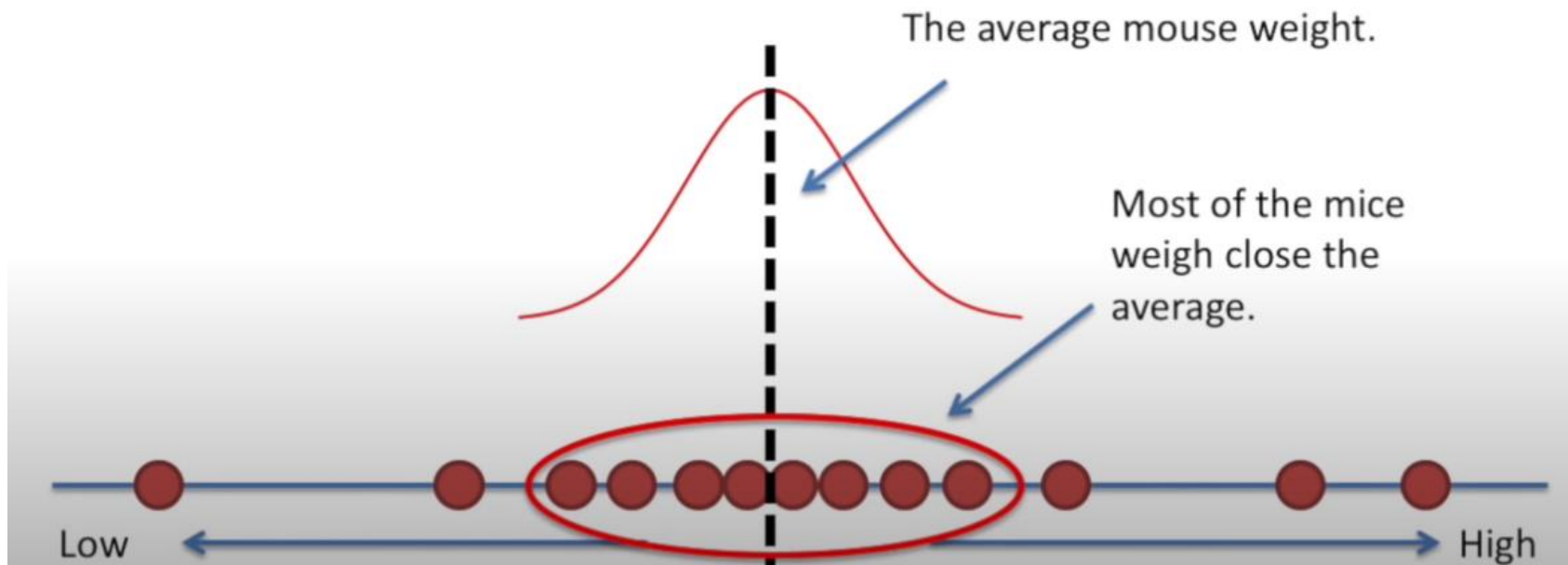
(i)

The reason you want to fit a distribution to your data is it can be easier to work with and it is also more general - it applies to every experiment of the same type.



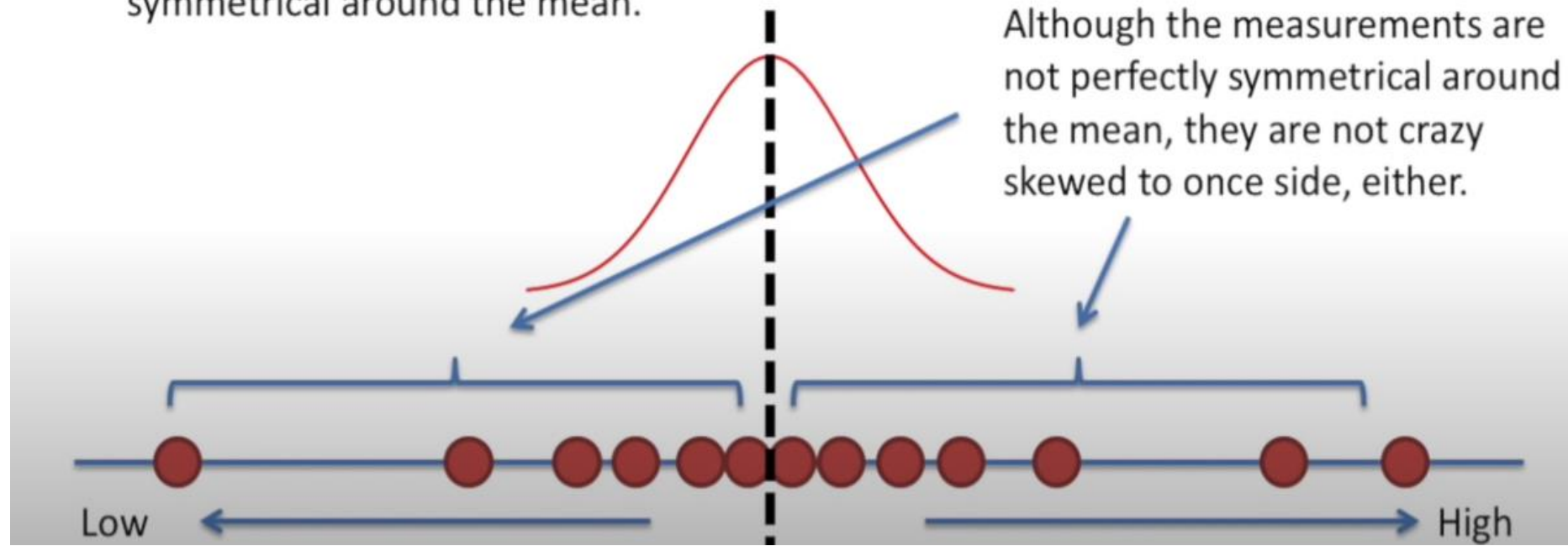
“Normally distributed” means a number of things:

- 1) We expect most of the measurements (mouse weights) to be close to the mean (average).

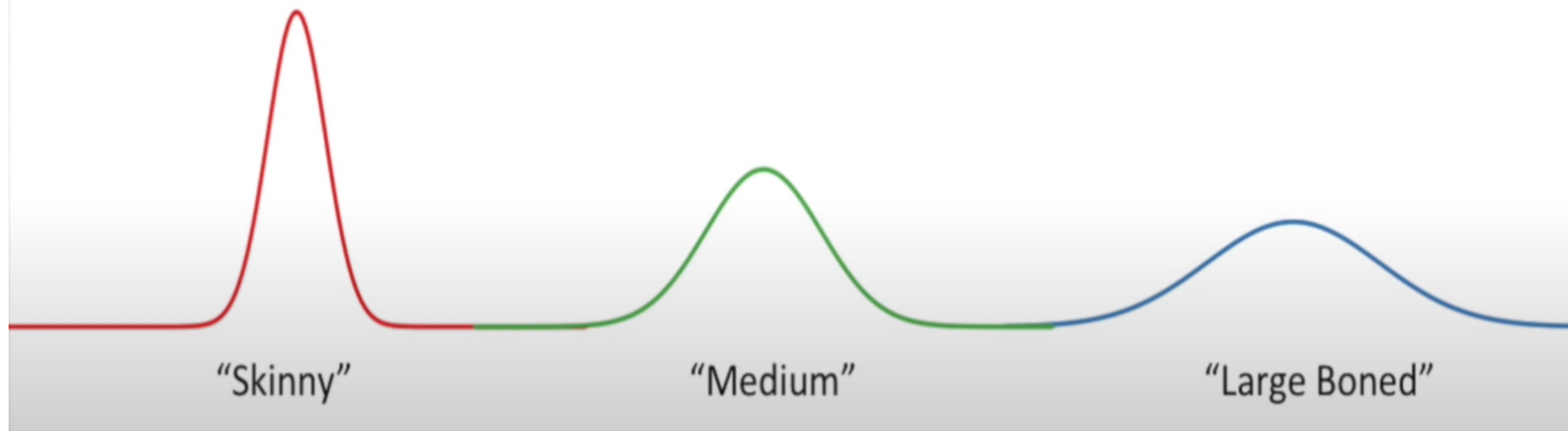


“Normally distributed” means a number of things:

- 1) We expect most of the measurements (mouse weights) to be close to the mean (average).
- 2) We expect the measurements to be relatively symmetrical around the mean.

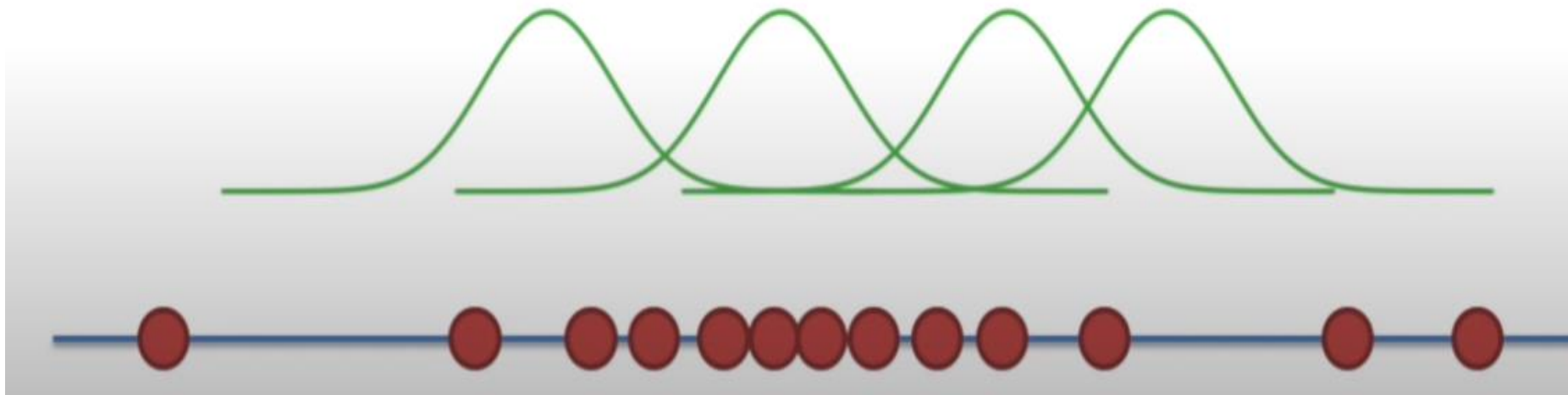


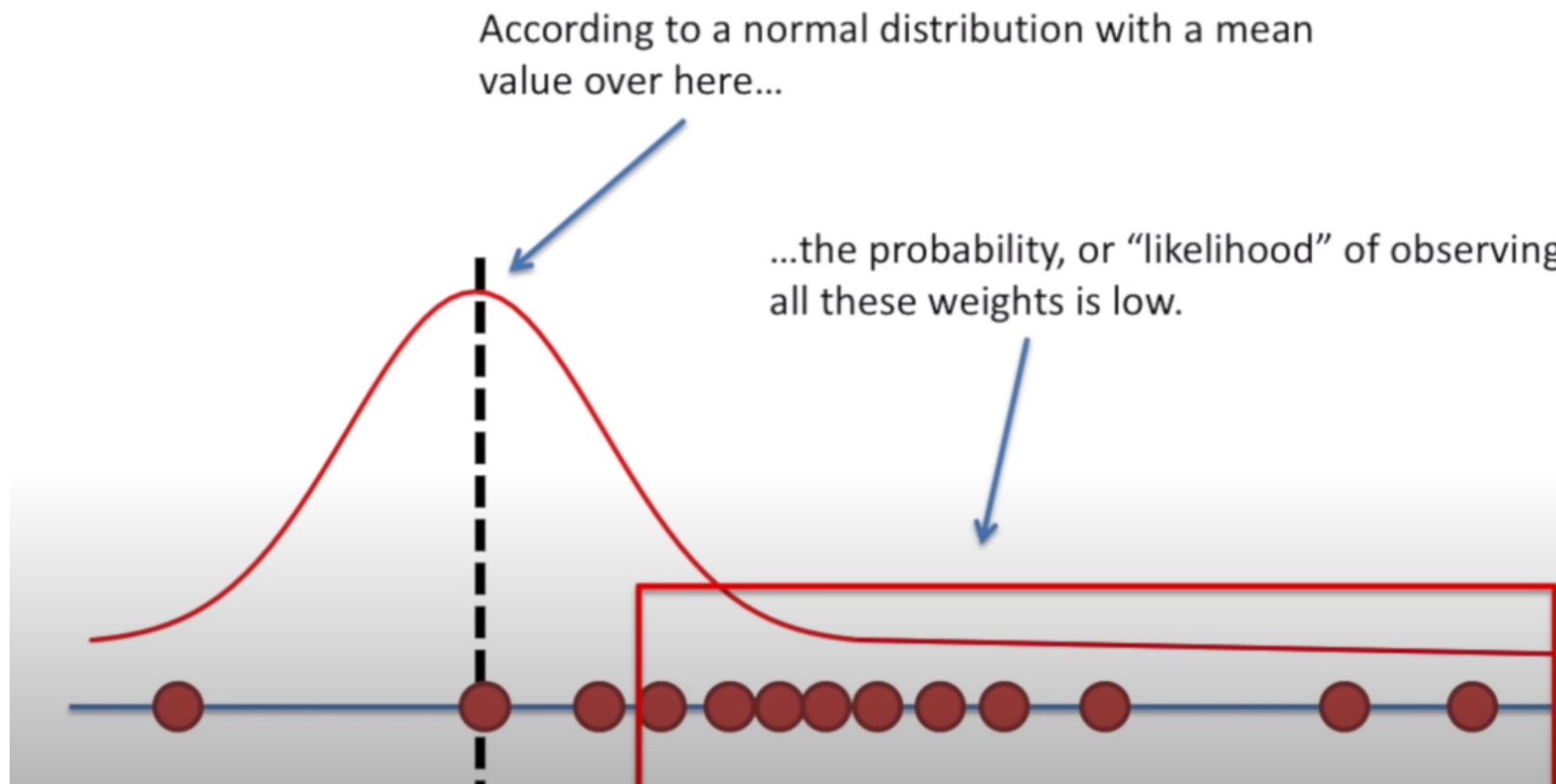
Normal distributions come in all kinds of shapes and sizes...



Once we settle on the shape, we have to figure out where to center the thing...

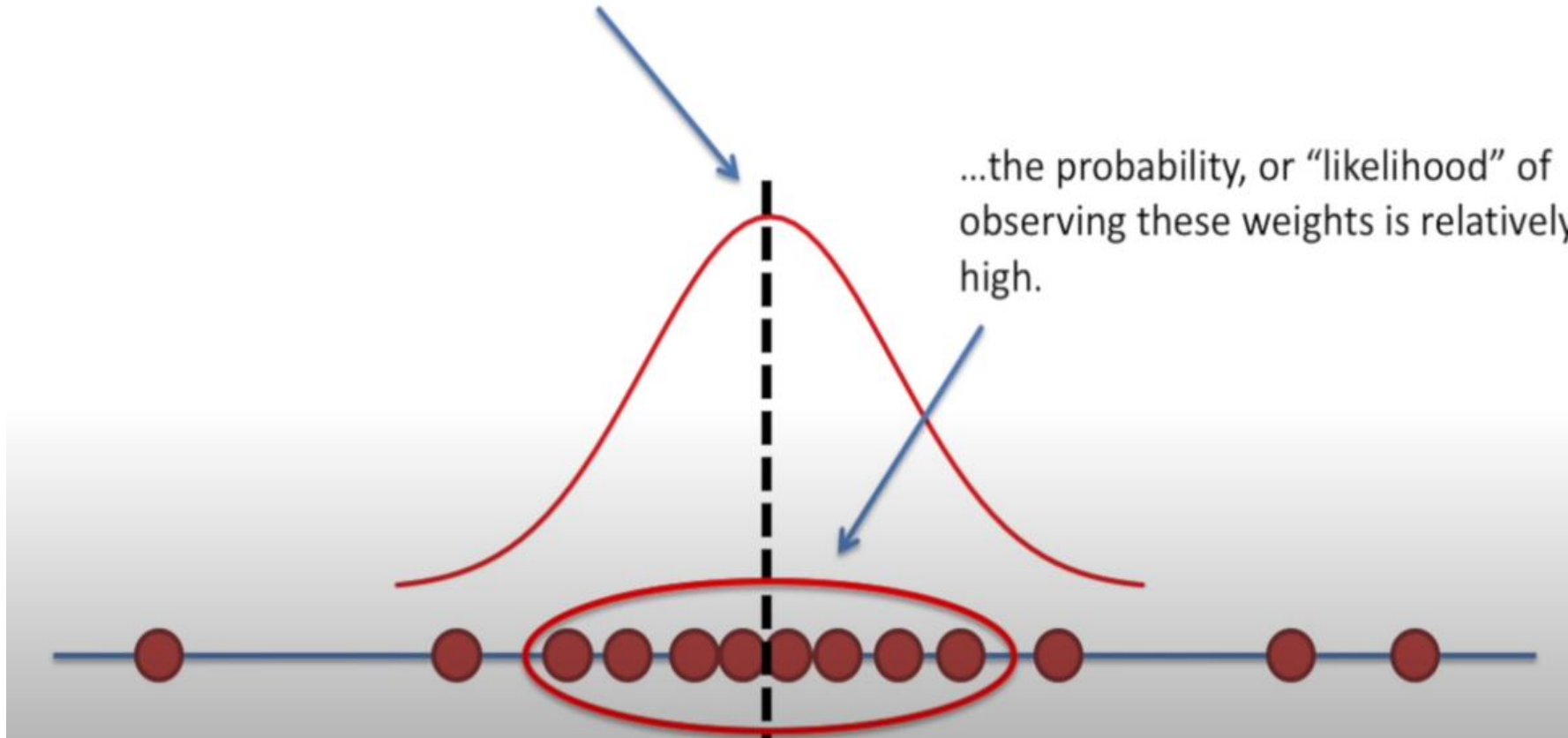
Is one location “better” than another?





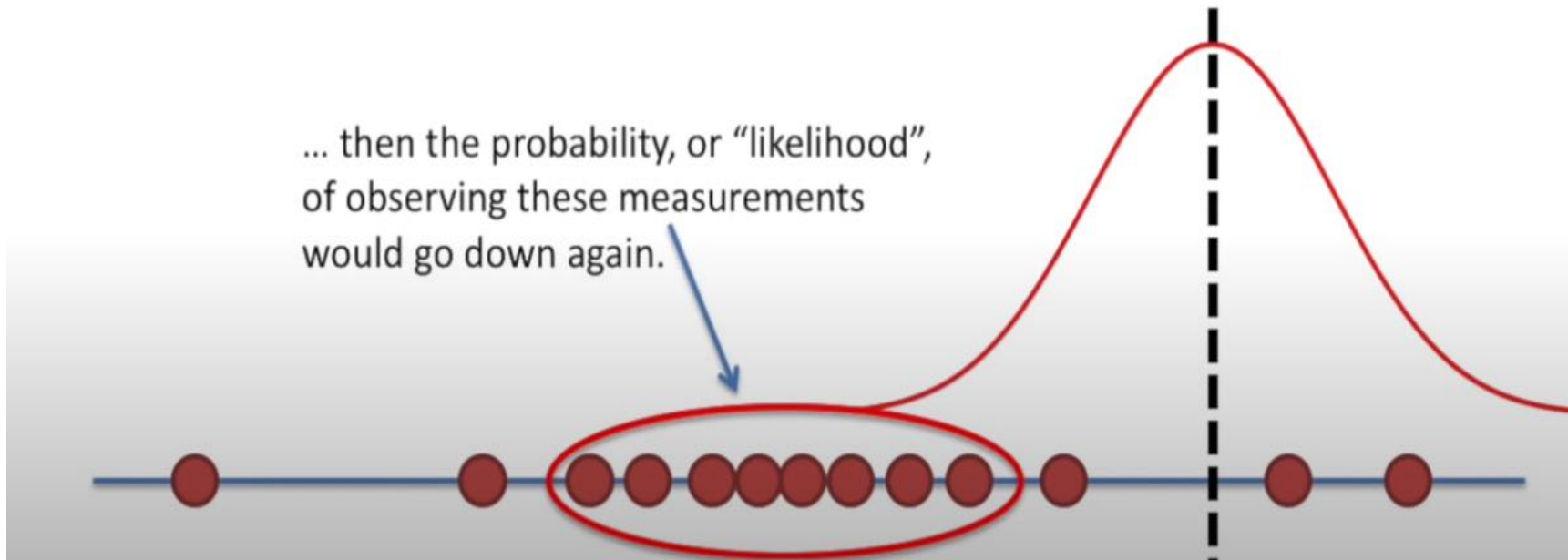
According to a normal distribution
with a mean value here...

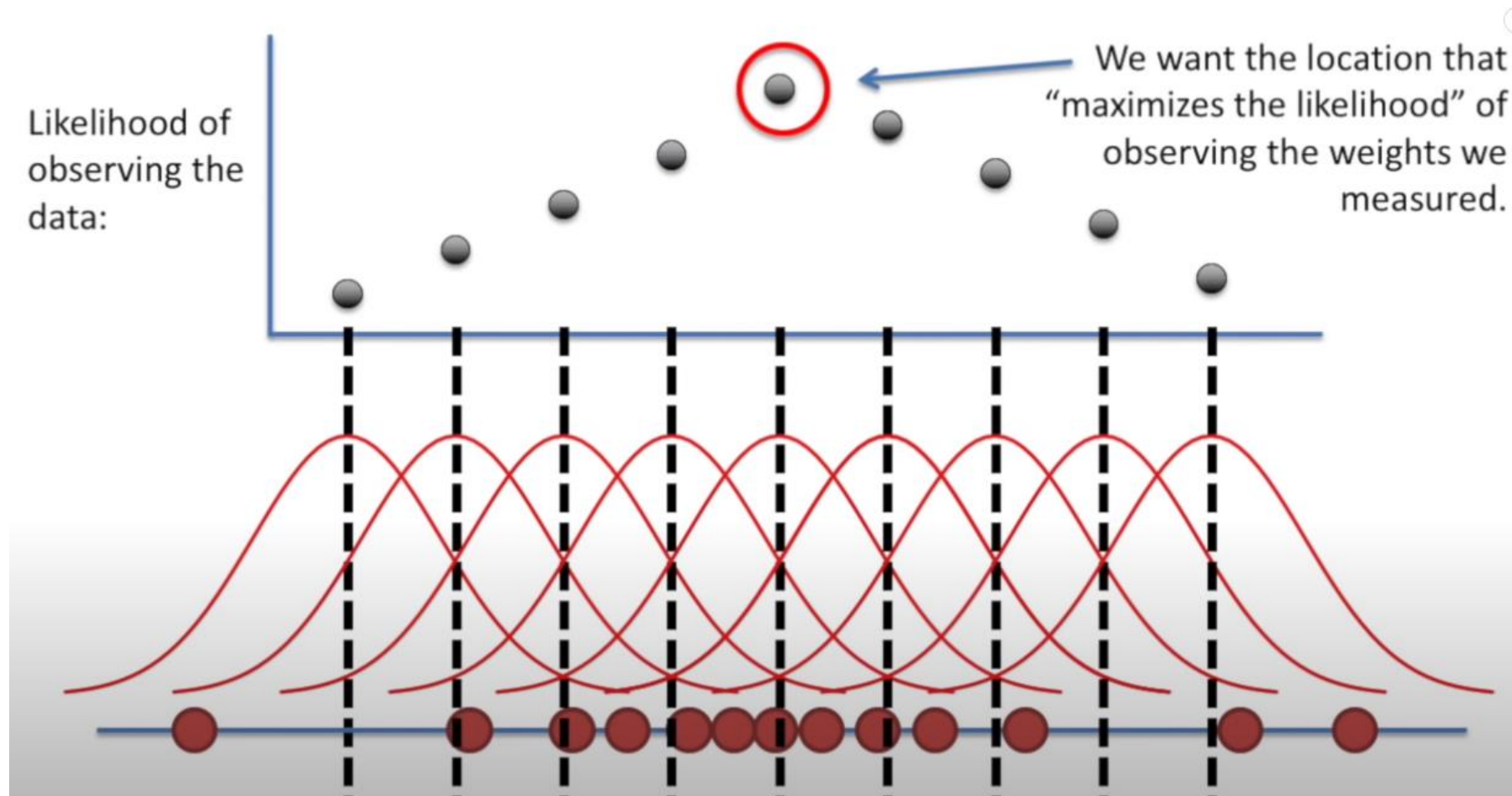
...the probability, or “likelihood” of
observing these weights is relatively
high.

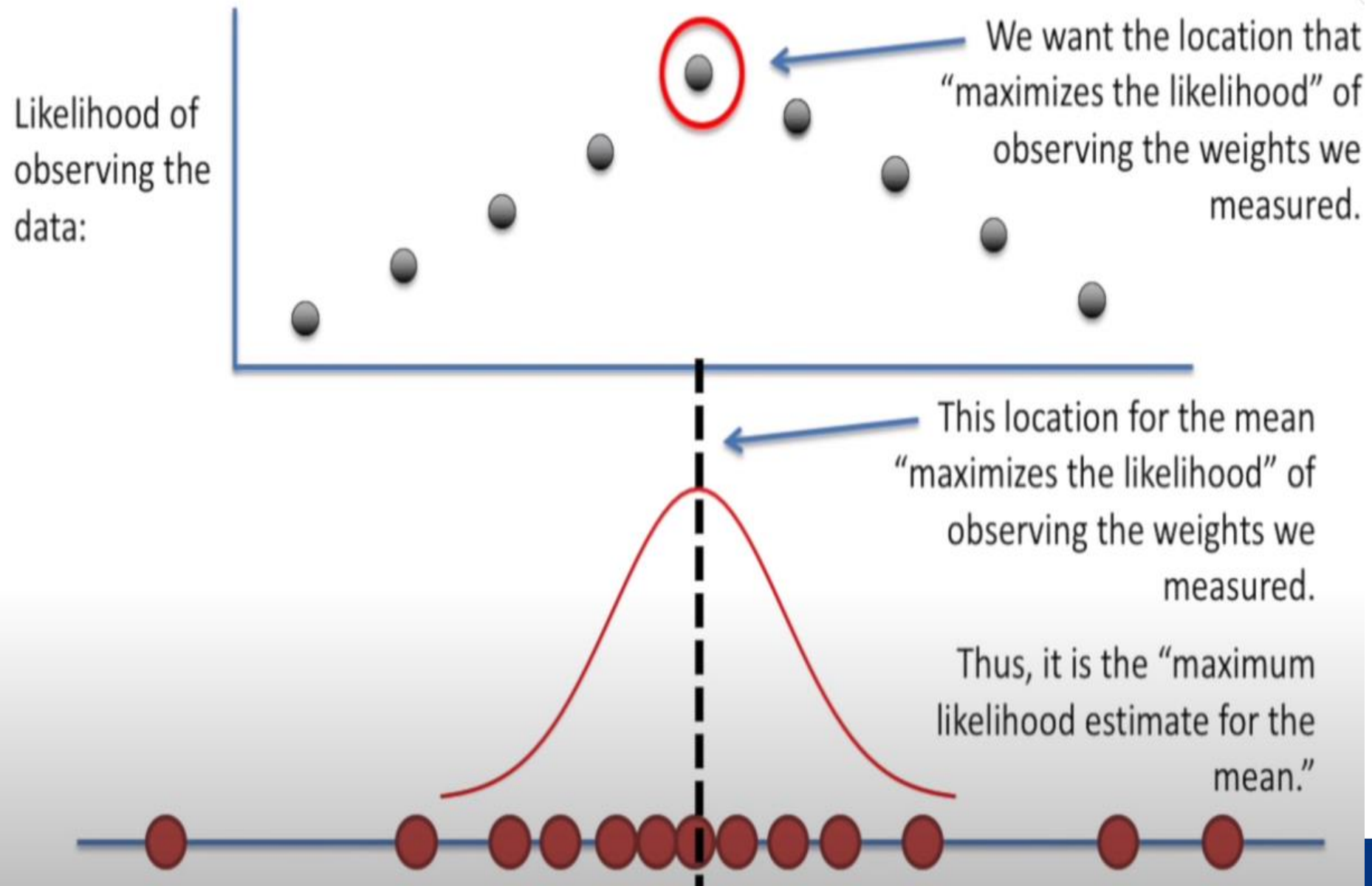


If we kept shifting the normal distribution over...

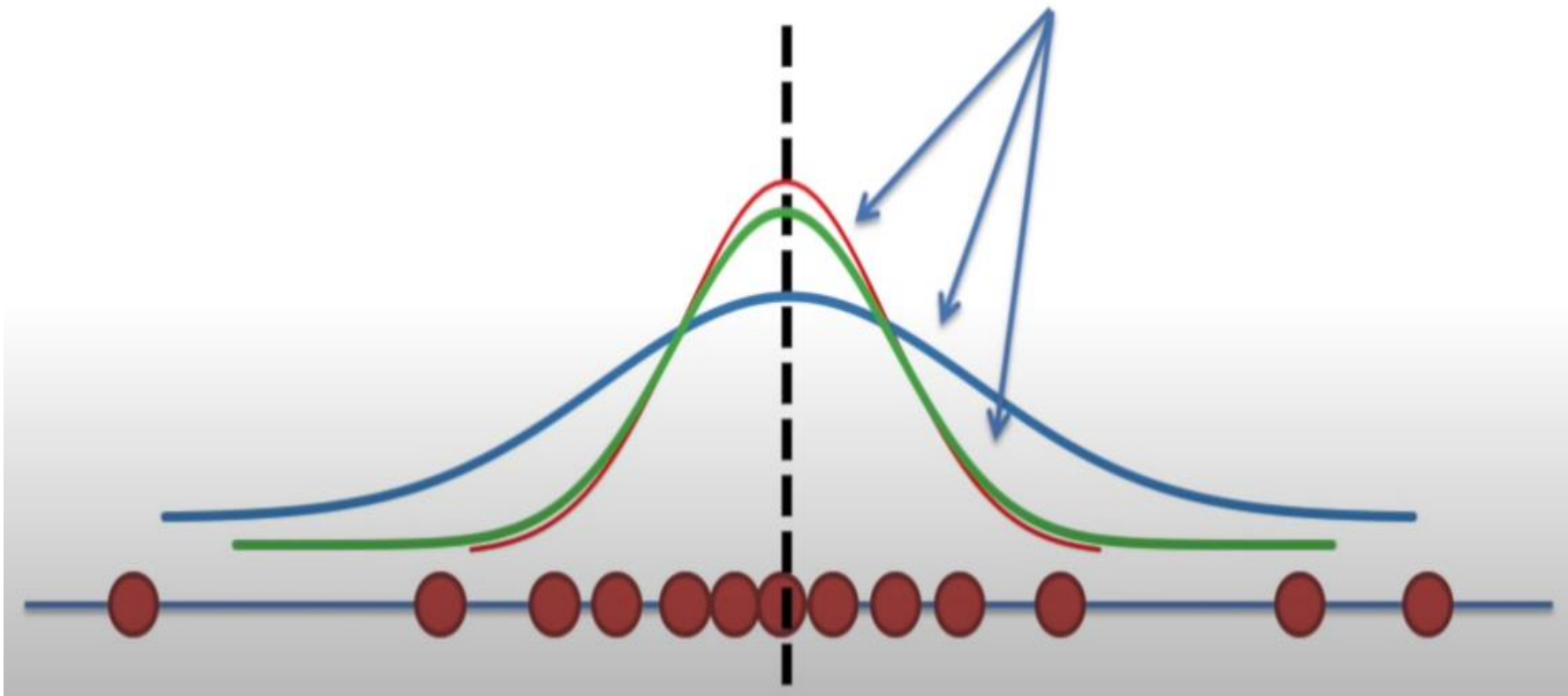
... then the probability, or “likelihood”, of observing these measurements would go down again.

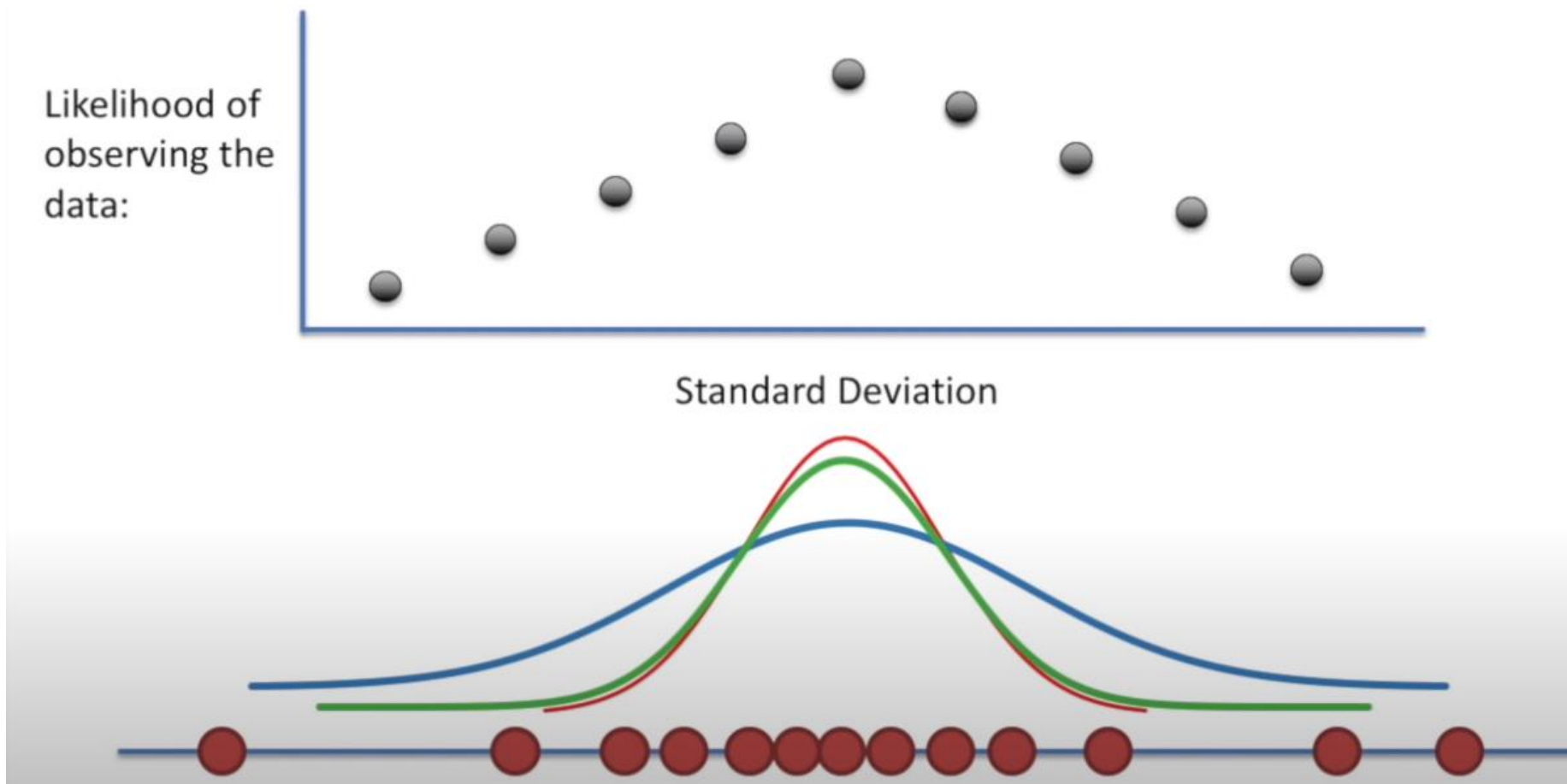


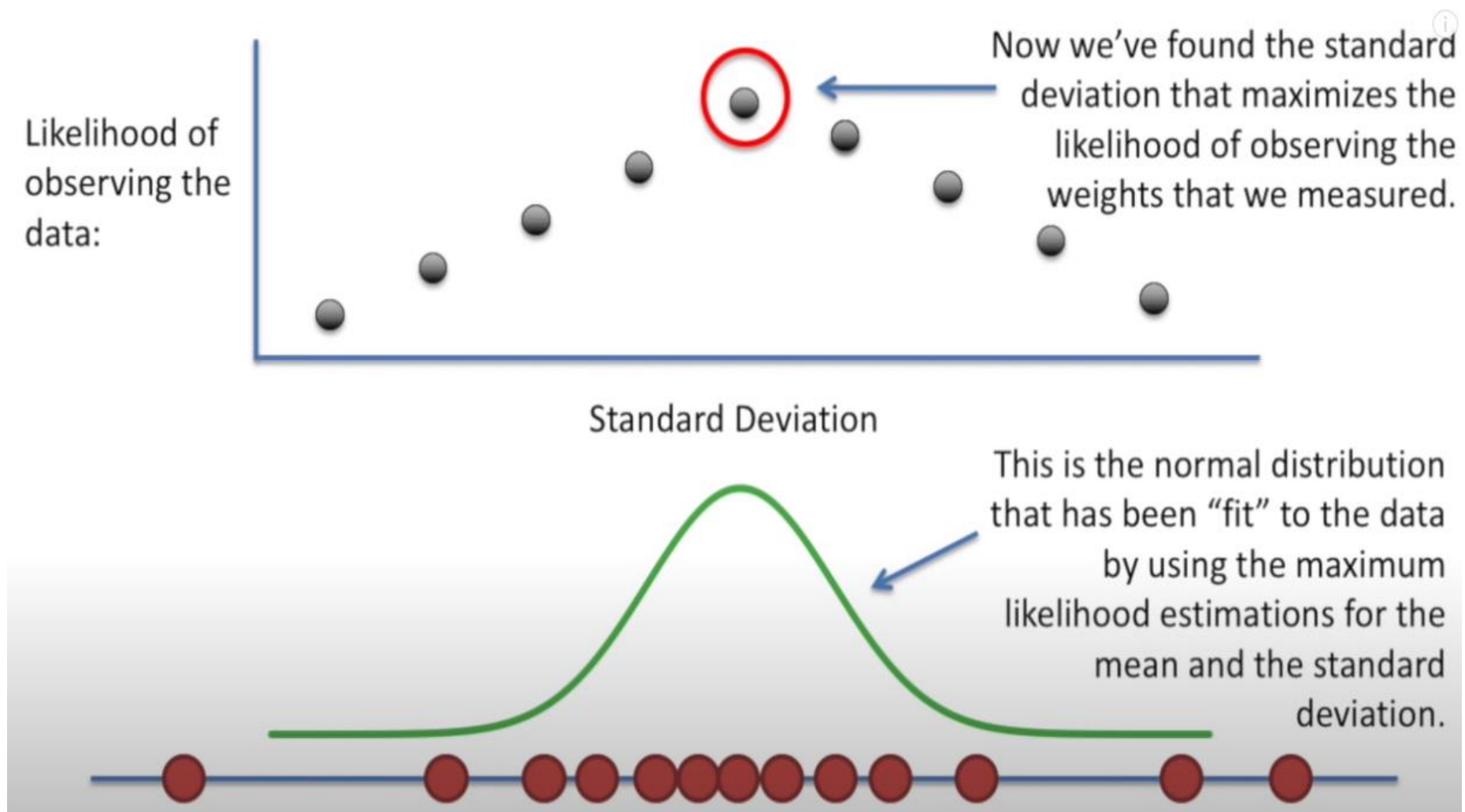




Now we have to figure out the
 “maximum likelihood estimate for
 the standard deviation...”







Fitting normal distribution: MLE

Algebraically: $\hat{\mu}, \hat{\sigma}^2 = \operatorname{argmax}_{\mu, \sigma^2} [Pr(x_{1...I} | \mu, \sigma^2)]$

where: $Pr(x | \mu, \sigma^2) = \text{Norm}_x[\mu, \sigma^2]$

or alternatively, we can maximize the logarithm

$$\begin{aligned}\hat{\mu}, \hat{\sigma}^2 &= \operatorname{argmax}_{\mu, \sigma^2} \left[\sum_{i=1}^I \log [\text{Norm}_{x_i}[\mu, \sigma^2]] \right] \\ &= \operatorname{argmax}_{\mu, \sigma^2} \left[-0.5I \log[2\pi] - 0.5I \log \sigma^2 - 0.5 \sum_{i=1}^I \frac{(x_i - \mu)^2}{\sigma^2} \right]\end{aligned}$$

Fitting normal distribution: MLE

$$\begin{aligned}\hat{\mu}, \hat{\sigma}^2 &= \operatorname{argmax}_{\mu, \sigma^2} \left[\sum_{i=1}^I \log [\operatorname{Norm}_{x_i}[\mu, \sigma^2]] \right] \\ &= \operatorname{argmax}_{\mu, \sigma^2} \left[-0.5I \log[2\pi] - 0.5I \log \sigma^2 - 0.5 \sum_{i=1}^I \frac{(x_i - \mu)^2}{\sigma^2} \right]\end{aligned}$$

How to maximize a function? Take derivative and set to zero.

$$\begin{aligned}\frac{\partial L}{\partial \mu} &= \sum_{i=1}^I \frac{(x_i - \mu)}{\sigma^2} \\ &= \frac{\sum_{i=1}^I x_i}{\sigma^2} - \frac{I\mu}{\sigma^2} = 0\end{aligned}$$

Solution:

$$\hat{\mu} = \frac{\sum_{i=1}^I x_i}{I}$$

Fitting normal distribution: MLE

Maximum likelihood solution:

$$\hat{\mu} = \frac{\sum_{i=1}^I x_i}{I}$$

$$\sigma^2 = \sum_{i=1}^I \frac{(x_i - \hat{\mu})^2}{I}$$

Should look familiar!

Maximum a posteriori (MAP)

What if we have some **prior knowledge** about θ ? Then, how should we change our knowledge about θ after observing data $\mathbf{x}$?

As the name suggests we find the parameters which maximize the posterior probability $Pr(\theta|\mathbf{x}_{1...I})$.

$$\begin{aligned}\hat{\theta} &= \operatorname{argmax}_{\theta} [Pr(\theta|\mathbf{x}_{1...I})] \\ &= \operatorname{argmax}_{\theta} \left[\frac{Pr(\mathbf{x}_{1...I}|\theta)Pr(\theta)}{Pr(\mathbf{x}_{1...I})} \right] \\ &= \operatorname{argmax}_{\theta} \left[\frac{\prod_{i=1}^I Pr(\mathbf{x}_i|\theta)Pr(\theta)}{Pr(\mathbf{x}_{1...I})} \right]\end{aligned}$$

Again we have assumed that data was **independent**

Maximum a posteriori (MAP)

Fitting

As the name suggests we find the parameters which maximize the posterior probability $Pr(\theta|\mathbf{x}_{1...I})$.

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \left[\frac{\prod_{i=1}^I Pr(\mathbf{x}_i|\theta)Pr(\theta)}{Pr(\mathbf{x}_{1...I})} \right]$$

Since the denominator doesn't depend on the parameters we can instead maximize

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \left[\prod_{i=1}^I Pr(\mathbf{x}_i|\theta)Pr(\theta) \right]$$

Maximum a posteriori

Predictive Density:

Evaluate new data point $\mathbf{x}^*$ under probability distribution with MAP parameters $Pr(\mathbf{x}^*|\hat{\boldsymbol{\theta}})$

Bayesian Approach

Fitting

Compute the posterior distribution over possible parameter values using Bayes' rule:

$$Pr(\boldsymbol{\theta} | \mathbf{x}_{1...I}) = \frac{\prod_{i=1}^I Pr(\mathbf{x}_i | \boldsymbol{\theta}) Pr(\boldsymbol{\theta})}{Pr(\mathbf{x}_{1...I})}$$

Principle: why pick one set of parameters? There are many values that could have explained the data. Try to capture all of the possibilities

Bayesian Approach

Predictive Density

- Each possible parameter value makes a prediction
- Some parameters more probable than others

$$Pr(\mathbf{x}^* | \mathbf{x}_{1...I}) = \int Pr(\mathbf{x}^* | \boldsymbol{\theta}) Pr(\boldsymbol{\theta} | \mathbf{x}_{1...I}) d\boldsymbol{\theta}$$

Make a prediction that is an infinite weighted sum (integral) of the predictions for each parameter value, where weights are the probabilities

Predictive densities for 3 methods

Maximum likelihood:

Evaluate new data point $\mathbf{x}^*$ under probability distribution with MLE parameter $Pr(\mathbf{x}^*|\hat{\theta})$

Maximum a posteriori:

Evaluate new data point $\mathbf{x}^*$ under probability distribution with MAP parameter $Pr(\mathbf{x}^*|\hat{\theta})$

Bayesian:

Calculate weighted sum of predictions from all possible values of parameters

$$Pr(\mathbf{x}^*|\mathbf{x}_{1...I}) = \int Pr(\mathbf{x}^*|\boldsymbol{\theta})Pr(\boldsymbol{\theta}|\mathbf{x}_{1...I}) d\boldsymbol{\theta}$$

Fitting normal distribution: MAP

Fitting

As the name suggests we find the parameters which maximize the posteriority $Pr(\theta | \mathbf{x}_{1..I})$..

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \left[\prod_{i=1}^I Pr(\mathbf{x}_i | \theta) Pr(\theta) \right]$$

Likelihood is normal PDF

$$Pr(x | \mu, \sigma^2) = \operatorname{Norm}_x[\mu, \sigma^2] = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-0.5 \frac{(x - \mu)^2}{\sigma^2} \right]$$

Conjugate prior

- **Definition.** A prior is **conjugate** for the likelihood function if the posterior is of the same form/type as the prior.
- Representative conjugate priors

Likelihood	Prior	Posterior
Poisson	Gamma	Gamma
Bernoulli	Beta	Beta
Binomial	Beta	Beta
Normal	Normal/inverse Gamma	Normal/inverse Gamma
Normal	Normal/inverse Wishart	Normal/inverse Wishart
Exponential	Gamma	Gamma
Multinomial	Dirichlet	Dirichlet

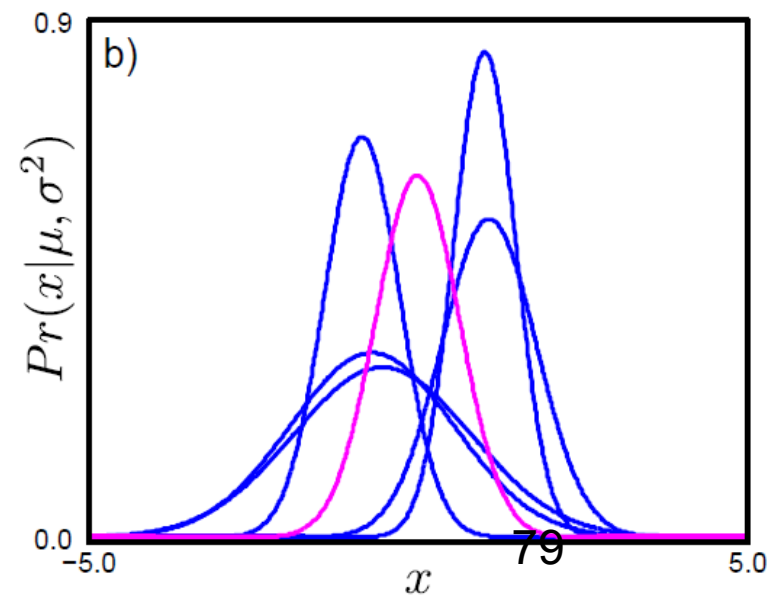
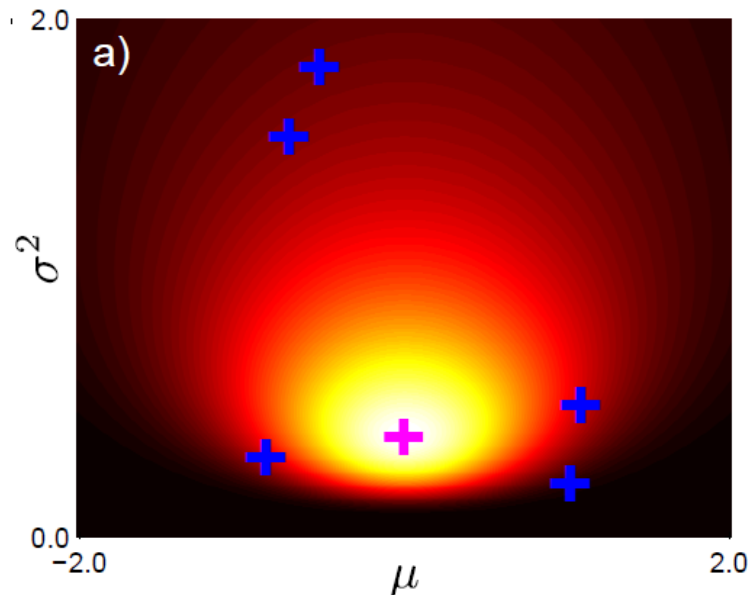
Fitting normal distribution: MAP

Prior

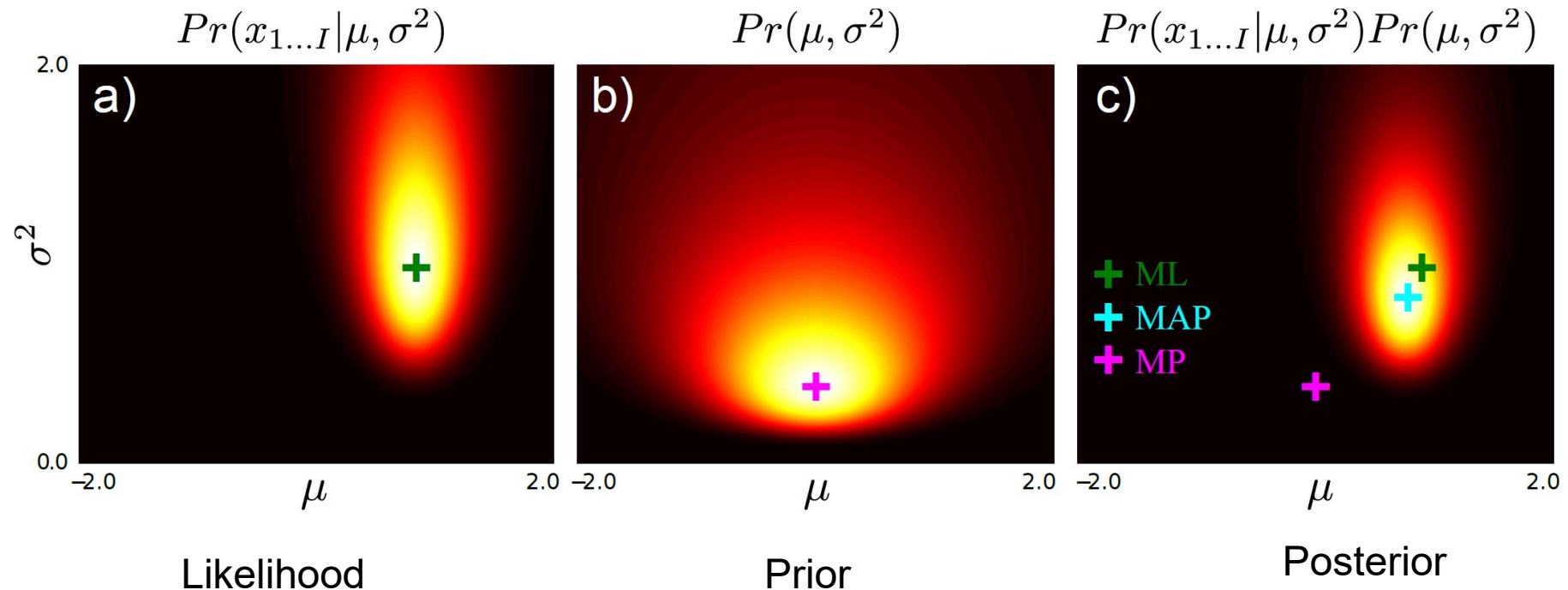
Use conjugate prior, normal scaled inverse gamma.

$$Pr(\mu, \sigma^2) = \text{NormInvGam}_{\mu, \sigma^2}[\alpha, \beta, \gamma, \delta]$$

$$Pr(\mu, \sigma^2) = \frac{\sqrt{\gamma}}{\sigma\sqrt{2\pi}} \frac{\beta^\alpha}{\Gamma[\alpha]} \left(\frac{1}{\sigma^2}\right)^{\alpha+1} \exp\left[-\frac{2\beta + \gamma(\delta - \mu)^2}{2\sigma^2}\right]$$



Fitting normal distribution: MAP



$$\begin{aligned}
 \hat{\mu}, \hat{\sigma}^2 &= \operatorname{argmax}_{\mu, \sigma^2} \left[\prod_{i=1}^I Pr(x_i | \mu, \sigma^2) Pr(\mu, \sigma^2) \right] \\
 &= \operatorname{argmax}_{\mu, \sigma^2} \left[\prod_{i=1}^I \operatorname{Norm}_{x_i}[\mu, \sigma^2] \operatorname{NormInvGam}_{\mu, \sigma^2}[\alpha, \beta, \gamma, \delta] \right]
 \end{aligned}$$

Fitting normal distribution: MAP

$$\begin{aligned}\hat{\mu}, \hat{\sigma}^2 &= \operatorname{argmax}_{\mu, \sigma^2} \left[\prod_{i=1}^I Pr(x_i | \mu, \sigma^2) Pr(\mu, \sigma^2) \right] \\ &= \operatorname{argmax}_{\mu, \sigma^2} \left[\prod_{i=1}^I \operatorname{Norm}_{x_i}[\mu, \sigma^2] \operatorname{NormInvGam}_{\mu, \sigma^2}[\alpha, \beta, \gamma, \delta] \right]\end{aligned}$$

Again maximize the log – does not change position of maximum

$$\hat{\mu}, \hat{\sigma}^2 = \operatorname{argmax}_{\mu, \sigma^2} \left[\sum_{i=1}^I \log[\operatorname{Norm}_{x_i}[\mu, \sigma^2]] + \log[\operatorname{NormInvGam}_{\mu, \sigma^2}[\alpha, \beta, \gamma, \delta]] \right]$$

Fitting normal distribution: MAP

MAP solution:

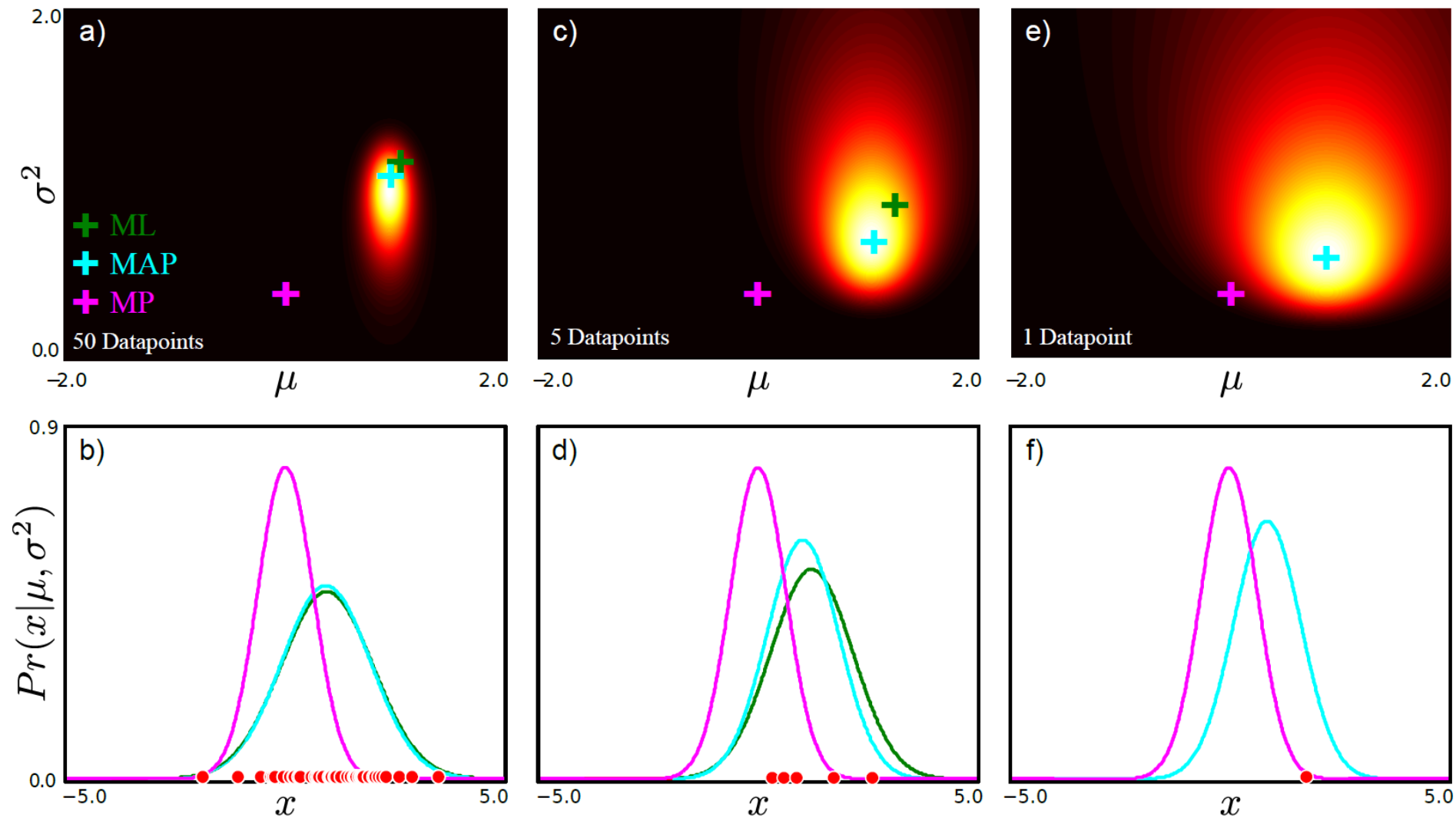
$$\hat{\mu} = \frac{\sum_{i=1}^I x_i + \gamma\delta}{I + \gamma}$$

$$\hat{\sigma}^2 = \frac{\sum_{i=1}^I (x_i - \mu)^2 + 2\beta + \gamma(\delta - \mu)^2}{I + 3 + 2\alpha}$$

Mean can be rewritten as weighted sum of data mean and prior mean:

$$\hat{\mu} = \frac{I\bar{x} + \gamma\delta}{I + \gamma}$$

Fitting normal distribution: MAP

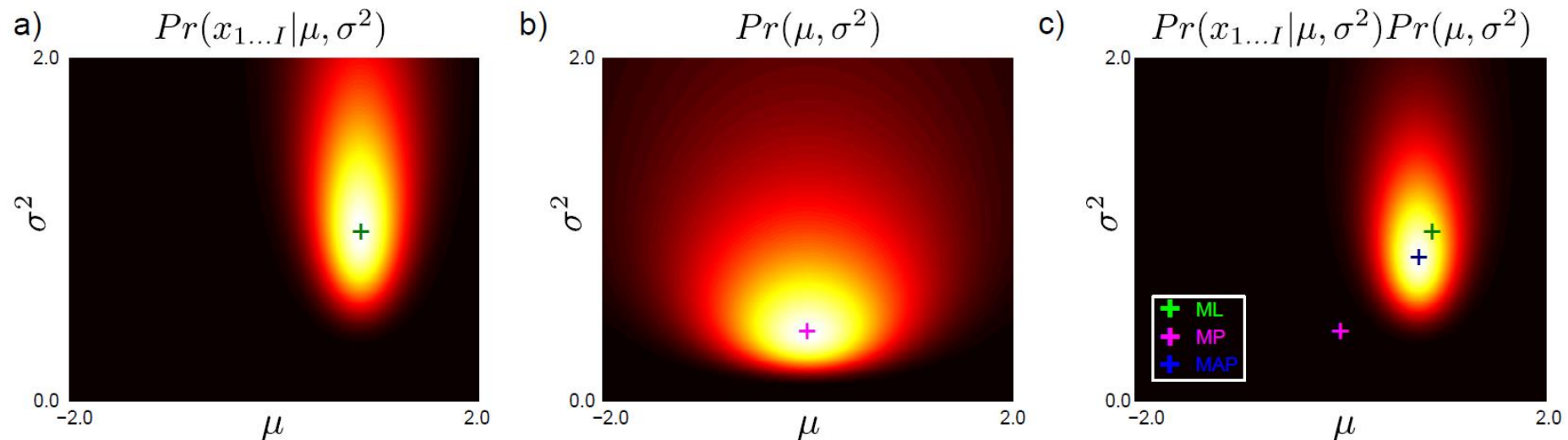


Fitting normal: Bayesian approach

Fitting

Compute the posterior distribution using Bayes' rule:

$$Pr(\boldsymbol{\theta}|\mathbf{x}_{1...I}) = \frac{\prod_{i=1}^I Pr(\mathbf{x}_i|\boldsymbol{\theta})Pr(\boldsymbol{\theta})}{Pr(\mathbf{x}_{1...I})}$$



Fitting normal: Bayesian approach

Fitting

Compute the posterior distribution using Bayes' rule:

$$\begin{aligned}
 Pr(\mu, \sigma^2 | x_{1...I}) &= \frac{\prod_{i=1}^I Pr(x_i | \mu, \sigma^2) Pr(\mu, \sigma^2)}{Pr(x_{1...I})} \\
 &= \frac{\prod_{i=1}^I \text{Norm}_{x_i}[\mu, \sigma^2] \text{NormInvGam}_{\mu, \sigma^2}[\alpha, \beta, \gamma, \delta]}{Pr(x_{1...I})} \\
 &= \frac{\kappa(\alpha, \beta, \gamma, \delta, x_{1...I}) \cdot \text{NormInvGam}_{\mu, \sigma^2}[\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}, \tilde{\delta}]}{Pr(x_{1...I})}, \\
 &= \text{NormInvGam}_{\mu, \sigma^2}[\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}, \tilde{\delta}].
 \end{aligned}$$

Two constants MUST cancel out or LHS not a valid pdf

Fitting normal: Bayesian approach

Fitting

Compute the posterior distribution using Bayes' rule:

$$Pr(\mu, \sigma^2 | x_{1...I}) = \text{NormInvGam}_{\mu, \sigma^2}[\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}, \tilde{\delta}]$$

where

$$\begin{aligned} \tilde{\alpha} &= \alpha + I/2, & \tilde{\gamma} &= \gamma + I & \tilde{\delta} &= \frac{(\gamma\delta + \sum_i x_i)}{\gamma + I} \\ \tilde{\beta} &= \frac{\sum_i x_i^2}{2} + \beta + \frac{\gamma\delta^2}{2} - \frac{(\gamma\delta + \sum_i x_i)^2}{2(\gamma + I)}. \end{aligned}$$

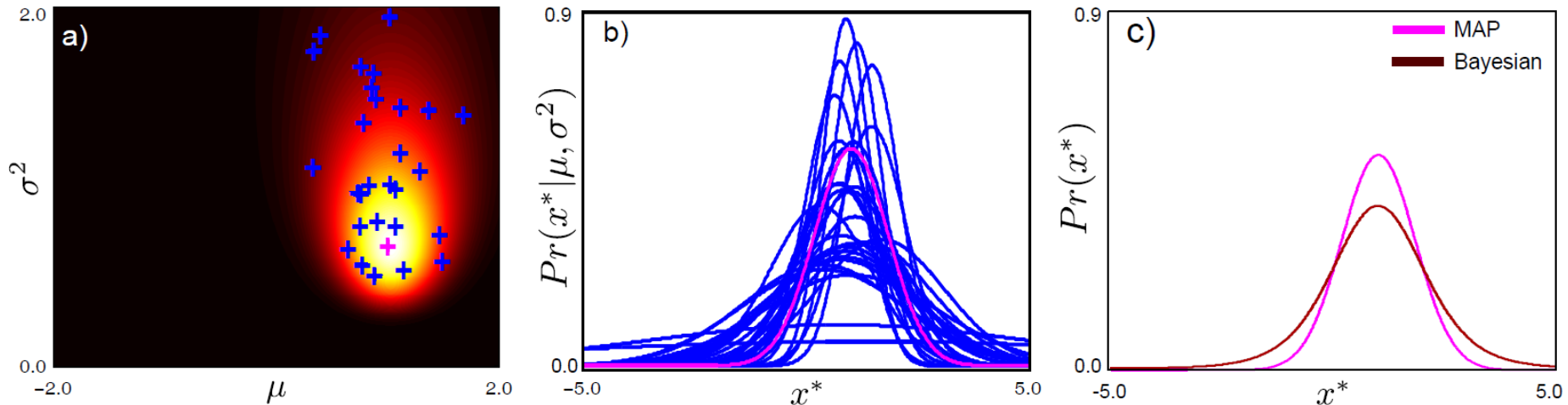
Fitting normal: Bayesian approach

Predictive density

Take weighted sum of predictions from different parameter values

$$Pr(x^*|x_{1...I}) = \iint Pr(x^*|\mu, \sigma^2) Pr(\mu, \sigma^2|x_{1...I}) d\mu d\sigma$$

The average of an infinite set of samples



Fitting normal: Bayesian approach

Predictive density

Take weighted sum of predictions from different parameter values:

$$\begin{aligned}
 Pr(x^*|x_{1...I}) &= \iint Pr(x^*|\mu, \sigma^2) Pr(\mu, \sigma^2|x_{1...I}) d\mu d\sigma \\
 &= \iint \text{Norm}_{x^*}[\mu, \sigma^2] \text{NormInvGam}_{\mu, \sigma^2}[\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}, \tilde{\delta}] d\mu d\sigma \\
 &= \iint \kappa(\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}, \tilde{\delta}, x_{1...I}) \cdot \text{NormInvGam}_{\mu, \sigma^2}[\check{\alpha}, \check{\beta}, \check{\gamma}, \check{\delta}] d\mu d\sigma \\
 &= \kappa(\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}, \tilde{\delta}, x_{1...I}) \iint \text{NormInvGam}_{\mu, \sigma^2}[\check{\alpha}, \check{\beta}, \check{\gamma}, \check{\delta}] d\mu d\sigma \\
 &= \kappa(\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}, \tilde{\delta}, x_{1...I})
 \end{aligned}$$

Fitting normal: Bayesian approach

Predictive density

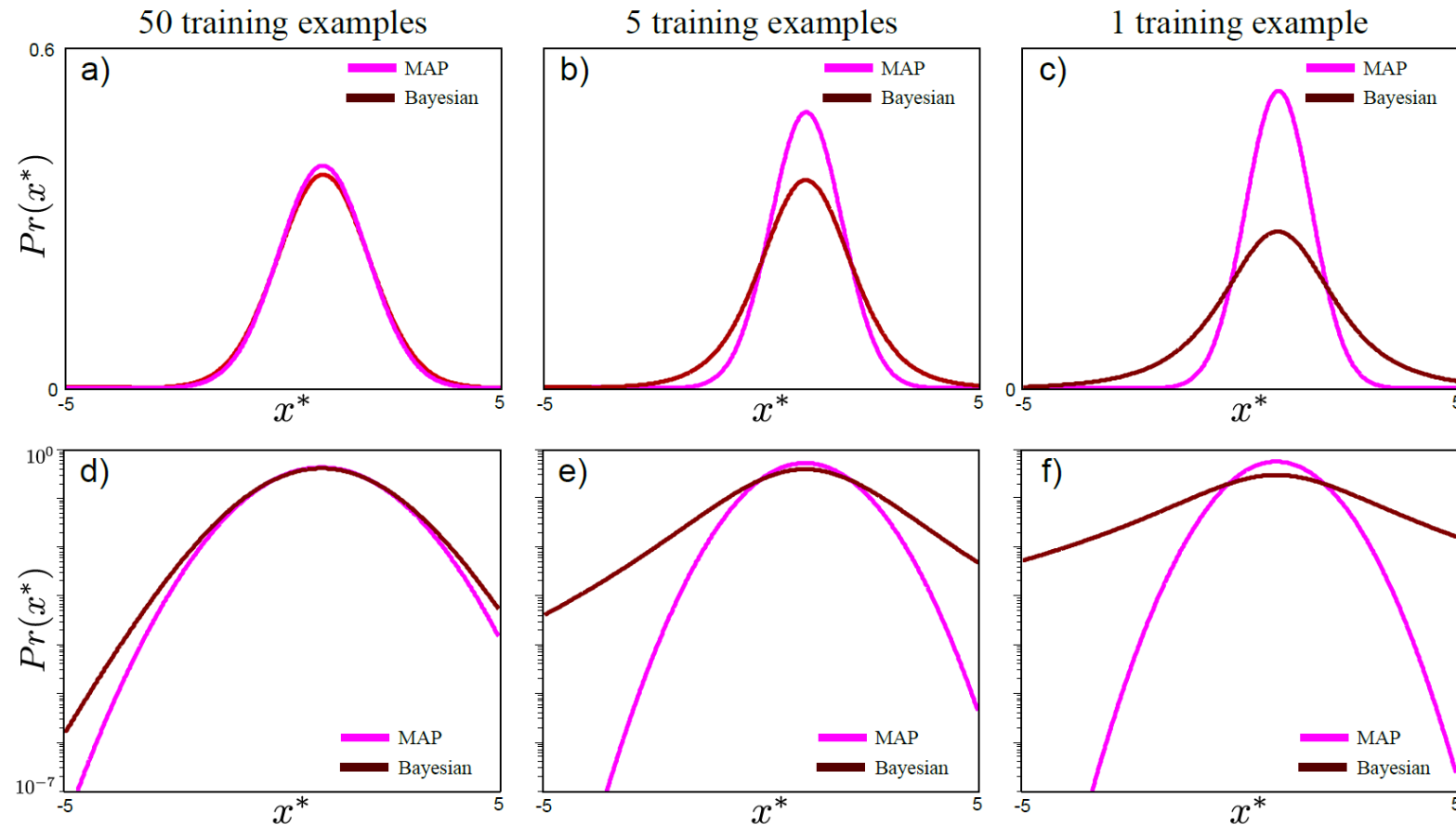
Take weighted sum of predictions from different parameter values:

$$Pr(x^*|x_{1...I}) = \kappa(\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}, \tilde{\delta}, x_{1...I}) = \frac{1}{\sqrt{2\pi}} \frac{\sqrt{\tilde{\gamma}} \tilde{\beta}^{\check{\alpha}}}{\sqrt{\check{\gamma}} \check{\beta}^{\check{\alpha}}} \frac{\Gamma[\check{\alpha}]}{\Gamma[\tilde{\alpha}]}$$

where

$$\begin{aligned} \check{\alpha} &= \tilde{\alpha} + 1/2, & \check{\gamma} &= \tilde{\gamma} + 1 \\ \check{\beta} &= \frac{x^{*2}}{2} + \tilde{\beta} + \frac{\tilde{\gamma}\tilde{\delta}^2}{2} - \frac{(\tilde{\gamma}\tilde{\delta} + x^*)^2}{2(\tilde{\gamma} + 1)}. \end{aligned}$$

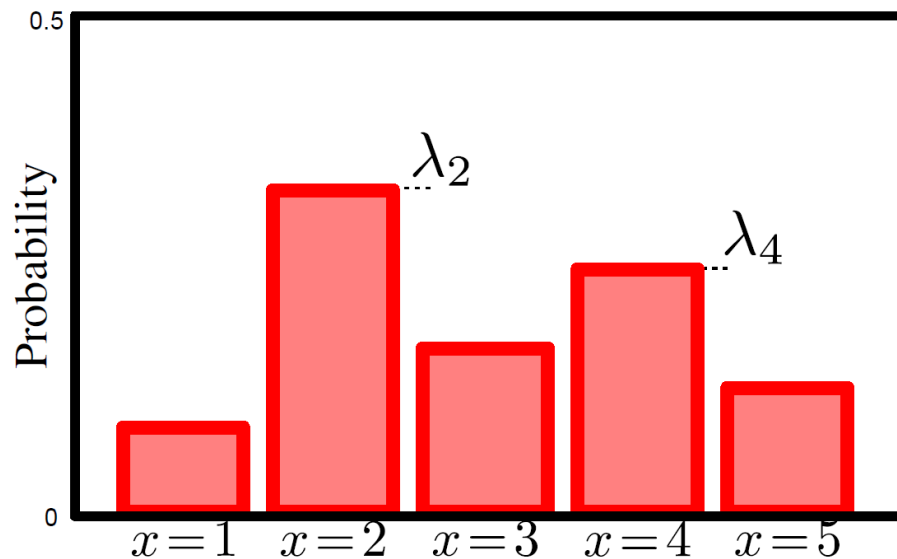
Fitting normal: Bayesian Approach



As the training data decreases, the Bayesian prediction becomes less certain but the MAP prediction is erroneously overconfident

Categorical distribution

Categorical Distribution



$$Pr(x = k) = \lambda_k$$

or can think of data as vector with all elements zero except k^{th} e.g. $[0,0,0,1,0]$

$$Pr(\mathbf{x} = \mathbf{e}_k) = \prod_{j=1}^K \lambda_j^{x_j} = \lambda_k$$

For short we write:

$$Pr(x) = \text{Cat}_x [\boldsymbol{\lambda}]$$

Categorical distribution describes situation where K possible outcomes $y=1 \dots y=k$.

Takes a K parameter $\lambda_k \in [0, 1]$

where $\sum_k \lambda_k = 1$

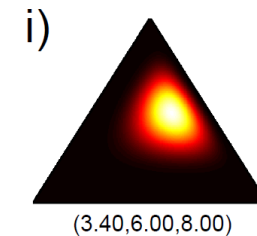
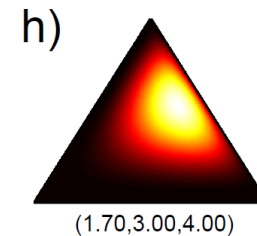
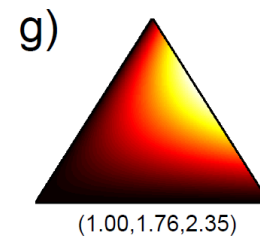
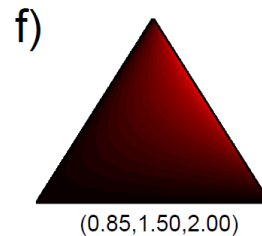
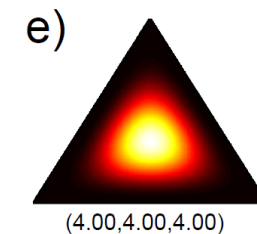
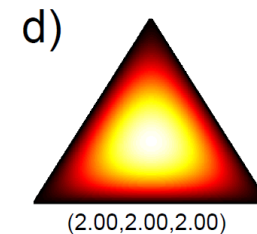
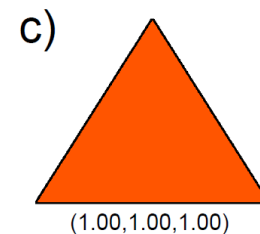
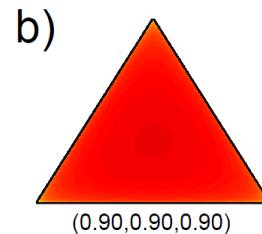
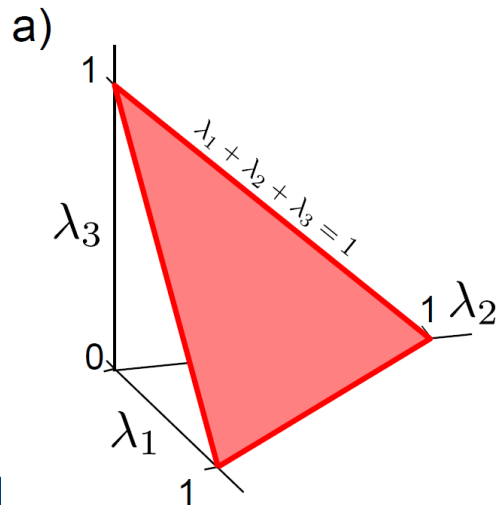
Dirichlet Distribution

Defined over K values $\lambda_k \in [0, 1]$ where $\sum_k \lambda_k = 1$

$$Pr(\lambda_1 \dots \lambda_K) = \frac{\Gamma[\sum_{k=1}^K \alpha_k]}{\prod_{k=1}^K \Gamma[\alpha_k]} \prod_{k=1}^K \lambda_k^{\alpha_k - 1}$$

Or for short: $Pr(\lambda_1 \dots \lambda_K) = \text{Dir}_{\lambda_1 \dots \lambda_K}[\alpha_1, \alpha_2, \dots, \alpha_K]$

Has k
parameters $\alpha_k > 0$



Categorical distribution: ML

$I=6$ simulates the roll of a dice

Maximize product of individual likelihoods

$$\begin{aligned}\hat{\lambda}_{1\dots 6} &= \operatorname{argmax}_{\lambda_{1\dots 6}} \left[\prod_{i=1}^I \operatorname{Pr}(x_i | \lambda_{1\dots 6}) \right] && \text{s.t. } \sum_k \lambda_k = 1 \\ &= \operatorname{argmax}_{\lambda_{1\dots 6}} \left[\prod_{i=1}^I \operatorname{Cat}_{x_i}[\lambda_{1\dots 6}] \right] && \text{s.t. } \sum_k \lambda_k = 1 \\ &= \operatorname{argmax}_{\lambda_{1\dots 6}} \left[\prod_{k=1}^6 \lambda_k^{N_k} \right] && \text{s.t. } \sum_k \lambda_k = 1\end{aligned}$$

N_k is the total number of times we observed the k -th bin

Categorical distribution: ML

Instead maximize the log probability

$$L = \sum_{k=1}^6 N_k \log[\lambda_k] + \nu \left(\sum_{k=1}^6 \lambda_k - 1 \right)$$

Log likelihood

Lagrange multiplier to
ensure that params sum to
one

Take derivative, set to zero and re-arrange:

$$\hat{\lambda}_k = \frac{N_k}{\sum_{m=1}^6 N_m}$$

The proportion of times we
observed bin k

Categorical distribution: MAP

MAP criterion:

$$\begin{aligned}
 \hat{\lambda}_{1\dots 6} &= \operatorname{argmax}_{\lambda_{1\dots 6}} \left[\prod_{i=1}^I Pr(x_i | \lambda_{1\dots 6}) Pr(\lambda_{1\dots 6}) \right] \\
 &= \operatorname{argmax}_{\lambda_{1\dots 6}} \left[\prod_{i=1}^I \operatorname{Cat}_{x_i} [\lambda_{1\dots 6}] \operatorname{Dir}_{\lambda_{1\dots 6}} [\alpha_{1\dots 6}] \right] \\
 &= \operatorname{argmax}_{\lambda_{1\dots 6}} \left[\prod_{k=1}^6 \lambda_k^{N_k} \prod_{k=1}^6 \lambda^{\alpha_k - 1} \right] \\
 &= \operatorname{argmax}_{\lambda_{1\dots 6}} \left[\prod_{k=1}^6 \lambda_k^{N_k + \alpha_k - 1} \right].
 \end{aligned}$$

Categorical distribution: MAP

Take derivative, set to zero and re-arrange:

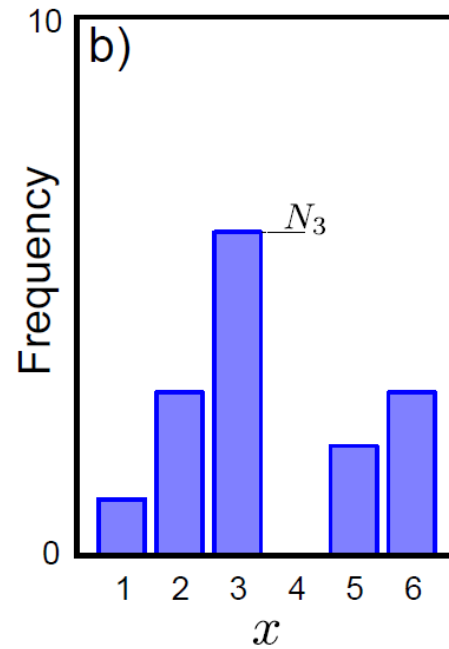
$$\hat{\lambda}_k = \frac{N_k + \alpha_k - 1}{\sum_{m=1}^6 (N_m + \alpha_m - 1)}$$

With a uniform prior ($\alpha_{1..K}=1$), gives same result as maximum likelihood.

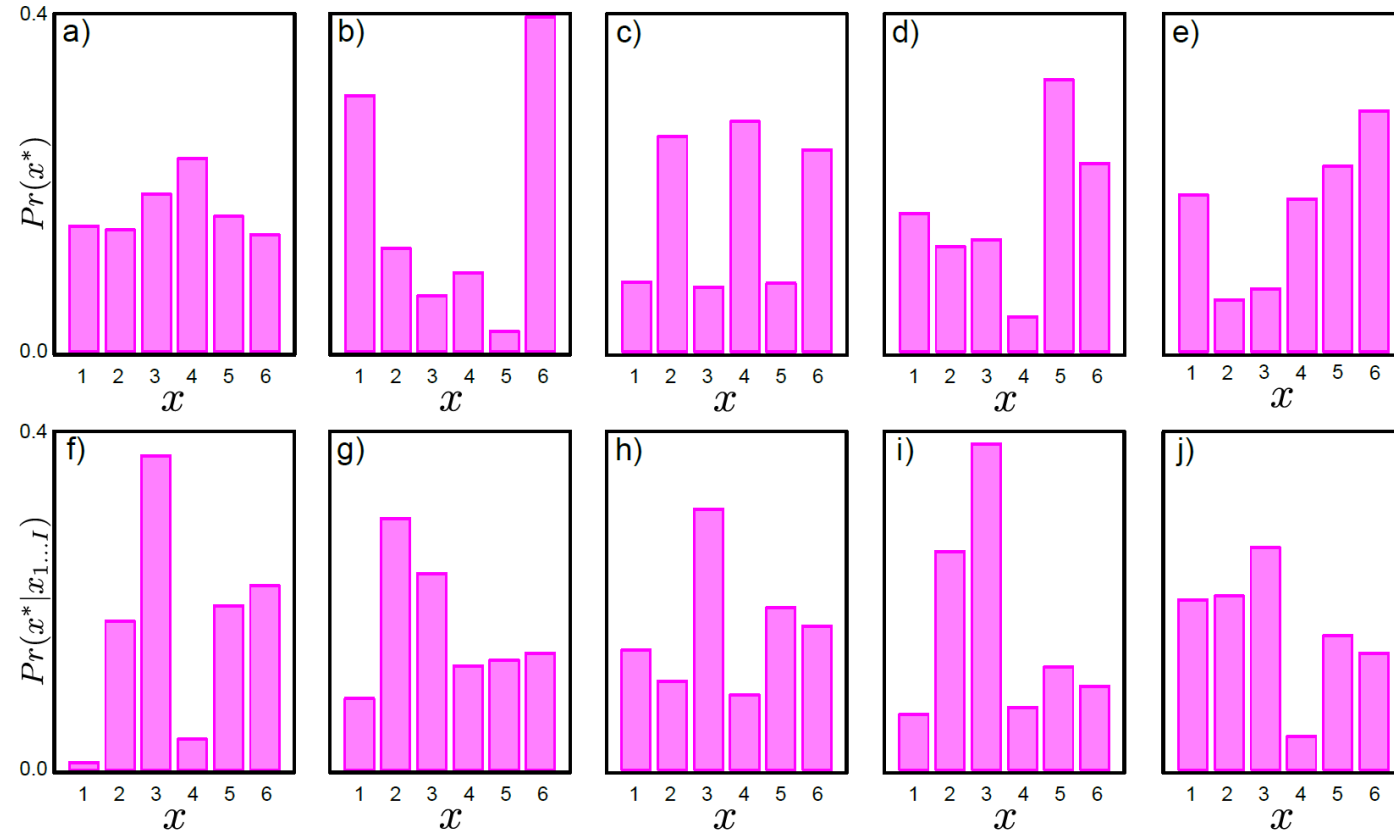
$$\hat{\lambda}_k = \frac{N_k}{\sum_{m=1}^6 N_m}$$

Categorical Distribution

Five samples from Dirichlet prior with equal α_i (uniform distribution)



Observed data



Five samples from posterior (MAP)

The distribution favors histograms where bin three is larger and bin four is small as suggested by the data.

Categorical Distribution: Bayesian approach

Compute posterior distribution over parameters:

$$\begin{aligned}
 Pr(\lambda_1 \dots \lambda_6 | x_{1\dots I}) &= \frac{\prod_{i=1}^I Pr(x_i | \lambda_{1\dots 6}) Pr(\lambda_{1\dots 6})}{Pr(x_{1\dots I})} \\
 &= \frac{\prod_{i=1}^I \text{Cat}_{x_i}[\lambda_{1\dots 6}] \text{Dir}_{\lambda_{1\dots 6}}[\alpha_{1\dots 6}]}{Pr(x_{1\dots I})} \\
 &= \frac{\kappa(\alpha_{1\dots 6}, x_{1\dots I}) \text{Dir}_{\lambda_{1\dots 6}}[\tilde{\alpha}_{1\dots 6}]}{Pr(x_{1\dots I})} \\
 &= \text{Dir}_{\lambda_{1\dots 6}}[\tilde{\alpha}_{1\dots 6}],
 \end{aligned}$$

Two constants MUST cancel out or LHS not a valid pdf

Categorical Distribution: Bayesian approach

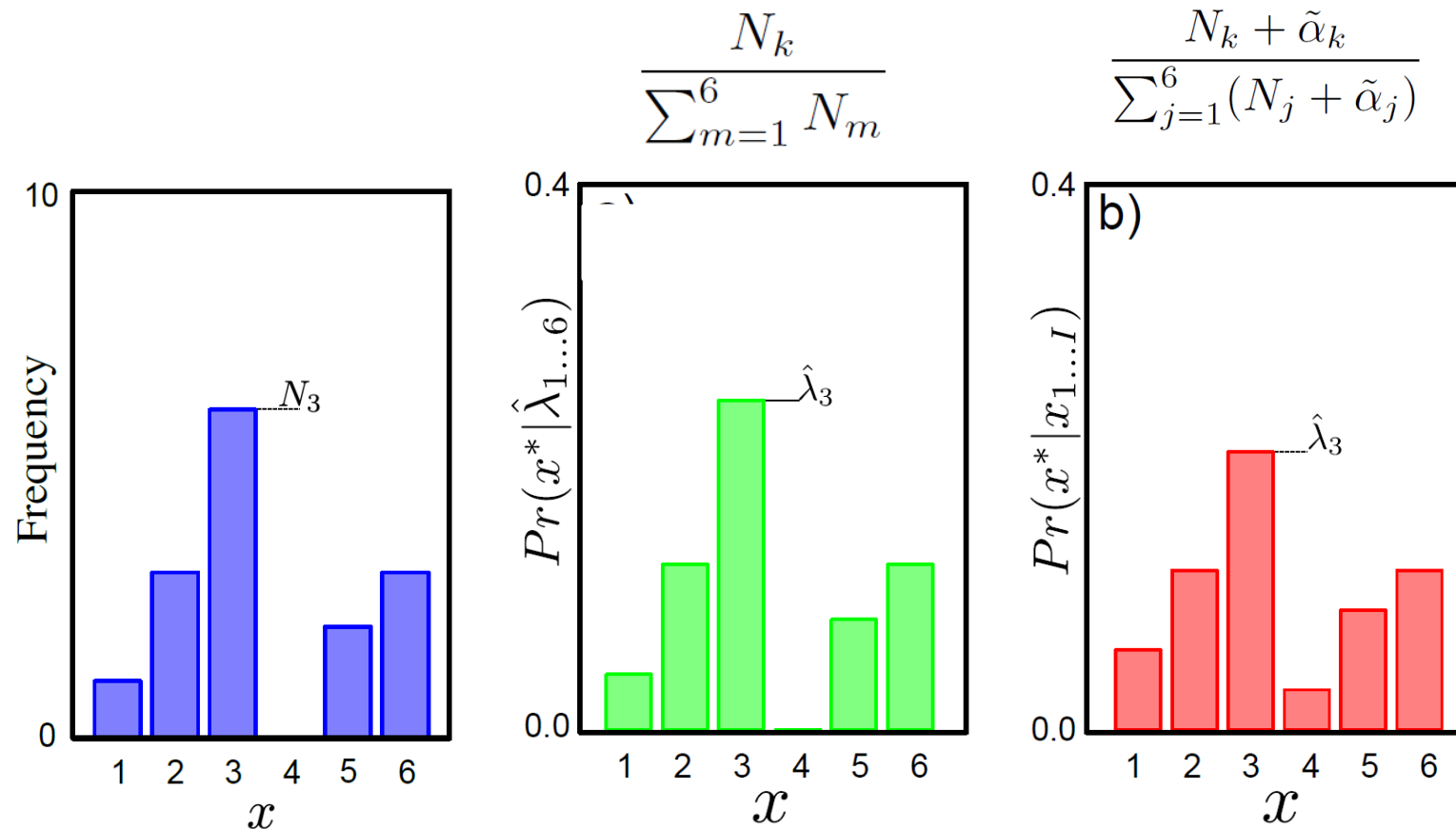
Compute predictive distribution:

$$\begin{aligned}
 Pr(x^*|x_{1...I}) &= \int Pr(x^*|\lambda_{1...6})Pr(\lambda_{1...6}|x_{1...I}) d\lambda_{1...6} \\
 &= \int \text{Cat}_{x^*}[\lambda_{1...6}]\text{Dir}_{\lambda_{1...6}}[\tilde{\alpha}_{1...6}] d\lambda_{1...6} \\
 &= \int \kappa(x^*, \tilde{\alpha}_{1...6})\text{Dir}_{\lambda_{1...6}}[\check{\alpha}_{1...6}] d\lambda_{1...6} \\
 &= \kappa(x^*, \tilde{\alpha}_{1...6}).
 \end{aligned}$$

Two constants MUST cancel out or LHS not a valid pdf

$$Pr(x^* = k|x_{1...I}) = \kappa(x^*, \tilde{\alpha}_{1...6}) = \frac{N_k + \tilde{\alpha}_k}{\sum_{j=1}^6 (N_j + \tilde{\alpha}_j)}$$

ML / MAP vs. Bayesian



The ML/MAP approaches are confident w.r.t. the observed data.
 The Bayesian approach predicts a more moderate distribution and allots some probability to the case $x = 4$ (we may have been unlucky in the data).

Conclusion

- Three ways to fit probability distributions
 - Maximum likelihood
 - Maximum a posteriori
 - Bayesian Approach
- Two worked example
 - Normal distribution (ML $\rightarrow$ least squares)
 - Categorical distribution